
韩先超老师 k8s 高薪架构师课程 CKA 和 CKS 精品班正在招生中，精品班课程提供技术+简历+面试指导+岗位内推，采用直播+录播模式进行全方位讲解。精品班包含国企真实项，除了 k8s 技术外，还包含安全渗透、go 开发 k8s 管理平台等

精品班课程详细地址：<https://edu.51cto.com/surl=VSmCT1>

韩先超老师 k8s 高薪架构师课程 CKA 和 CKS 精品班正在招生中，精品班课程提供技术+简历+面试指导+岗位内推，采用直播+录播模式进行全方位讲解。精品班包含国企真实项，除了 k8s 技术外，还包含安全渗透、go 开发 k8s 管理平台等

精品班课程详细地址：<https://edu.51cto.com/surl=VSmCT1>

一、韩老师 51CTO 平台视频号地址（定期发布免费技术视频，随时观看）：

<https://edu.51cto.com/lecturer/12964849-b6.html>

二、韩先超老师微信公众号，定期发布技术类文章（运维、开发、测试、DBA、网络、渗透安全、K8S 等均有）



三、韩老师gitee地址，会更新一些k8s、云原生等项目：

<https://gitee.com/hanxianchao66>

四、韩先超老师微信：

luckylucky421302

或者扫描下方二维码添加：

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

韩先超-15011572657
中国大连



扫一扫上面的二维码图案，加我为好友。

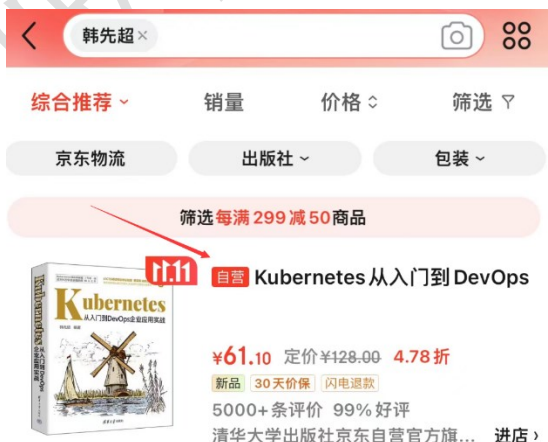
加微信，备注是k8s学员

五、韩老师抖音号：打开抖音搜索 hxck8s1，抖音定期开直播，发布免费视频，免费分享技术：如 Linux、Docker、K8S、DBA、安全渗透、网路等，扫描下方二维码也可以关注：



六、韩老师是清华大学出版社出版的【Kubernetes 从入门到 DevOps 企业应用实战】畅销书的作者，权威专家，京东和抖音等平台均可购买，大家如果平时玩抖音，可以通过抖音平台购买，能在抖音视频评论区留言交流，如果不玩抖音的，可以在京东平台购买：

购买方式一： 京东平台购买，打开京东，搜索韩先超，找到自营店购买或者打开链接直接购买
-> <https://item.jd.com/13853531.html>



版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

购买方式二：通过抖音平台可购买，打开抖音，搜索抖音号：hxck8s，关注之后进到商品橱窗即可购买。



Docker 官网: <https://docs.docker.com/>

Docker 的 github 地址: <https://github.com/moby/moby>

Dockerhub 官网

<https://registry.hub.docker.com>

如果 docker 官方 registry 拉取镜像速度很慢，可以尝试 daocloud 提供的加速器服务

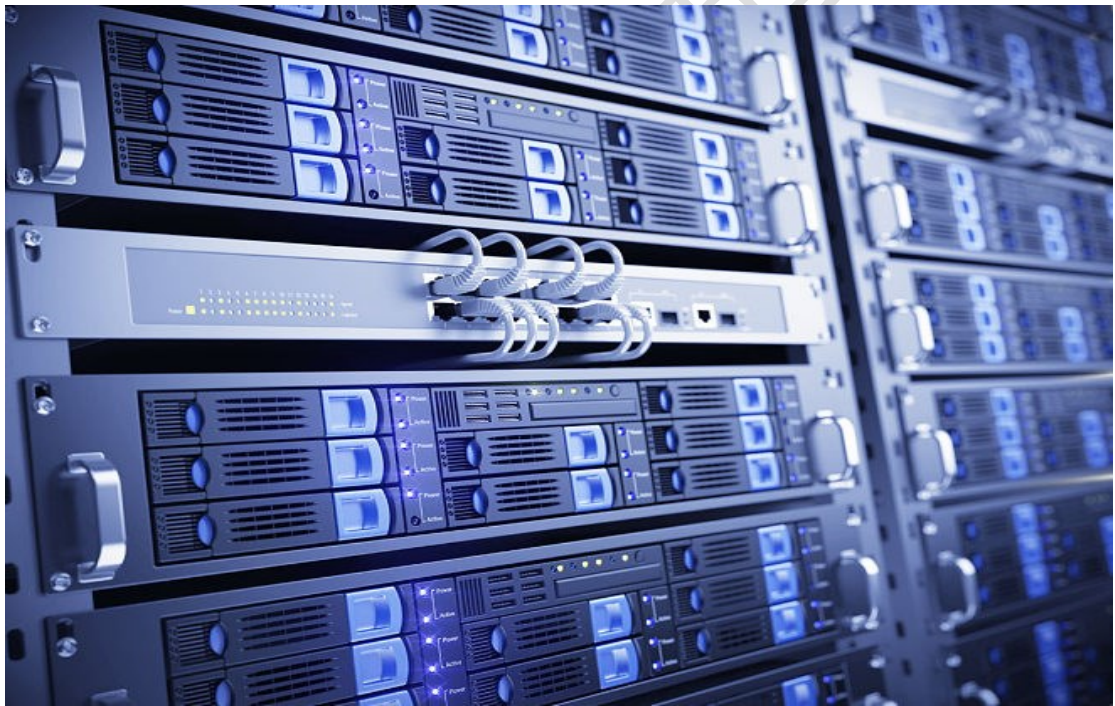
<https://dashboard.daocloud.io/mirror>

linux 服务器、虚拟机和容器的区别：

1、linux 服务器

服务器就是在机房里放的有完整操作系统的机器，Linux 是一个操作系统，linux 服务器就和 Windows 的服务器是一样的，都是在网络环境或分布式处理环境中，为用户提供服务，可分为访问服务器、文件服务器、数据库服务器、通信服务器和应用服务器等。

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。



常见的 Linux 服务器的系统主要包括 debian 系（debian、ubuntu）、redhat 系(redhat、centos)，suse、rockylinux、麒麟、欧拉等，同一类发行版用法相似。

Linux 服务器的优点：

- 1) 提供稳定性，因为基于 Linux 的服务器不容易崩溃。在遇到碰撞的情况下，整个系统都不受影响。
- 2) Linux 非常适合阻止或防止可疑恶意软件进入并影响整个系统的性能。
- 3) 拥有耐用性，因为它可以长时间运行，一年不关机也不会影响性能。
- 4) 大多数 Linux 发行版是免费下载的，可以安装在自己个人电脑上

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

2、虚拟机和容器

什么是虚拟机？

虚拟机（VM）指通过软件模拟的具有完整硬件系统功能的、运行在一个完全隔离环境中的完整计算机系统。简单讲是可以实现在一台物理计算机上模拟多台计算机运行任务。

常见的虚拟机软件：

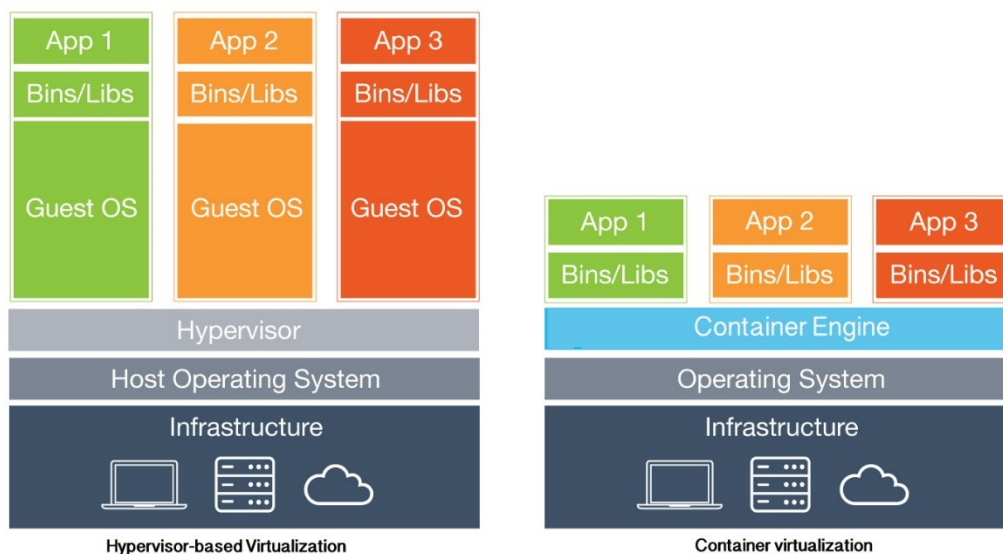
Vmware Workstation、VirtualBox、KVM、Hyper-V、Xen 等

优点：

- 1、一台主机，通过安装不同系统环境的虚拟机、与运行单独的实体服务器相比是经济的。
- 2、虚拟机与主机操作系统隔离，**虚拟机上的操作不影响主机**

什么是容器？

容器是一种沙盒技术，主要目的是为了将应用运行在其中，与外界隔离；及方便这个沙盒可以被转移到其它宿主机上。通俗点的理解就是一个装应用程序的箱子，箱子里面有软件运行所需的依赖库和配置。开发人员可以把这个箱子搬到任何机器上，且不影响里面软件的运行。



CSDN @WHQ556677

上图左侧是虚拟机的架构。可以看出，有 4 个操作系统 (Operating System)，分别是 Host Operating System 和 3 个 Guest Operating System。每个虚拟机中都有一个独立的 Kernel。

右侧为容器架构，很明显图中只有一个操作系统，即只有一个独立的 Kernel。

App 应用在虚拟机中运行时，调用需要经过虚拟化软件的处理，才能调用到真正的宿主机操作系统。

在容器中运行，直接可以使用宿主机操作系统，省事省力。

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

备注：hypervisor

Hypervisor，又称虚拟机监视器（英语：virtual machine monitor，缩写为 VMM），是用来建立与执行虚拟机器的软件、固件或硬件。被 Hypervisor 用来执行一个或多个虚拟机器的电脑称为**主体机器**（host machine），这些虚拟机器则称为客体机器（guest machine）。hypervisor 提供虚拟的作业平台来执行客体操作系统（guest operating systems），负责管理其他客体操作系统的执行阶段；这些客体操作系统，共同分享虚拟化后的硬件资源。

物理机、虚拟机、容器总结：

虚拟机中需要模拟一台物理机的所有资源，比如你要模拟出有多少 CPU、显卡等等，这些都是在软件层面通过计算资源实现的，容器仅在操作系统层面上，对所需各类资源进行了隔离。容器具备敏捷和高性能：直接调用宿主机操作系统，不需要经过虚拟化软件的处理。

容器是在操作系统之上，每个容器共享操作系统内核，执行文件和库等，意味着一台物理机仅安装一个操作系统可以运行多个容器。容器是非常轻量的，几秒即可启动。相比容器，VM 需要几分钟启动。这意味着可以在同一个服务器上相比 VM 运行 2-3 倍多的应用，并且，容器能帮助创建一个可移植的，一致的开发测试部署环境。

容器的思想来自于集装箱，集装箱解决了什么问题？在一艘大船上，可以把货物规整的摆放起来。并且各种各样的货物被集装箱标准化了，集装箱和集装箱之间不会互相影响。那么我就不需要专门运送水果的船和专门运送化学品的船了。只要这些货物在集装箱里封装的好好的，那我就可以用一艘大船把他们都运走。

服务器就好比大船。容器就是集装箱。

1、不同的应用程序可能会有不同的应用环境，比如 .net 开发的网站和 php 开发的网站依赖的软件就不一样，如果把他们依赖的软件都安装在一个服务器上就要调试很久，而且很麻烦，还会造成一些冲突。比如 IIS 和 Apache 访问端口冲突。这个时候你就要隔离 .net 开发的网站和 php 开发的网站。常规来讲，我们可以在服务器上创建不同的虚拟机在不同的虚拟机上放置不同的应用，但是虚拟机开销比较高。docker 可以实现虚拟机隔离应用环境的功能，并且开销比虚拟机小，小就意味着省钱了。

2、你开发软件的时候用的是 Ubuntu，但是运维管理的都是 centos，运维在把你的软件从开发环境转移到生产环境的时候就会遇到一些 Ubuntu 转 centos 的问题，比如：有个特殊版本的数据库，只有 Ubuntu 支持，centos 不支持，在转移的过程当中运维就得想办法解决这样的问题。这时候要是有了 docker 你就可以把开发环境直接封装转移给运维，运维直接部署你给他的 docker 就可以了（就问你不问）。而且部署速度快。

为什么学习 Docker？

Docker 就是容器的一个代表，相当于把服务打包到 docker 容器里。

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

一个 Java 代码，需要通过 mvn 编译，编译源码之后会生成 war 包或者 jar 包，可以把 jar 包启动，就可以对外提供服务了（能正常访问页面，做操作）。如果是 war 包，需要弄到 tomcat 服务的 webapp 目录下，去启动。

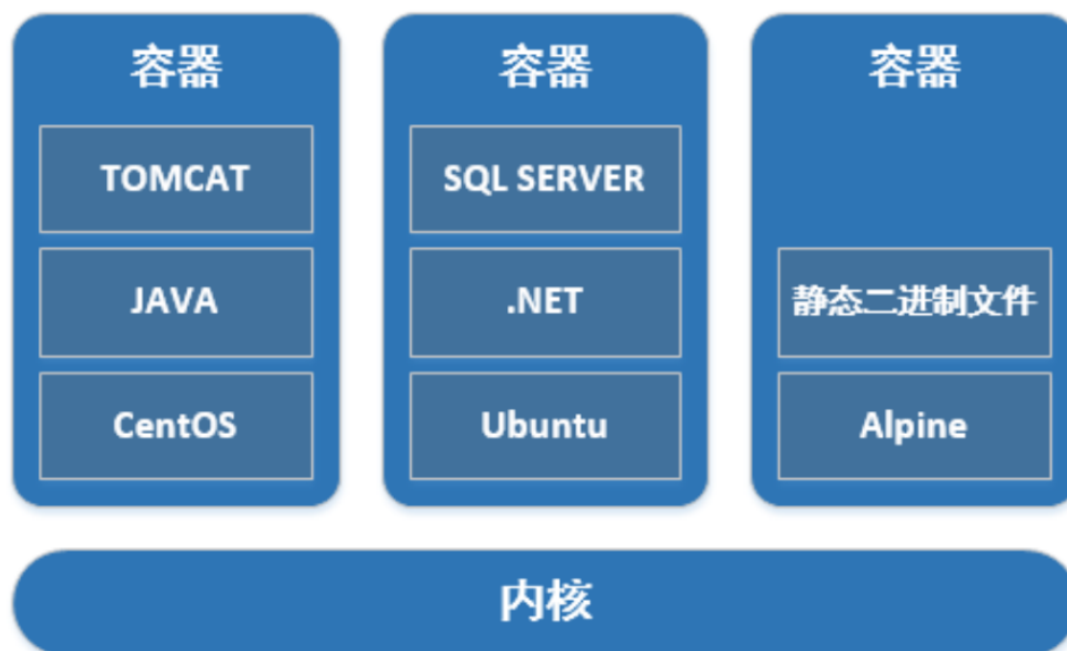
没有用容器之前通常会出现如下问题：

- 1.开发环境没问题，测试环境报错---->排查后发现有启动脚本里 jdk 版本跟代码要求的版本不匹配
- 2.测试环境没问题，生产环境报错 -->排查发现有 tomcat 版本和 jdk 版本不匹配。

然后：

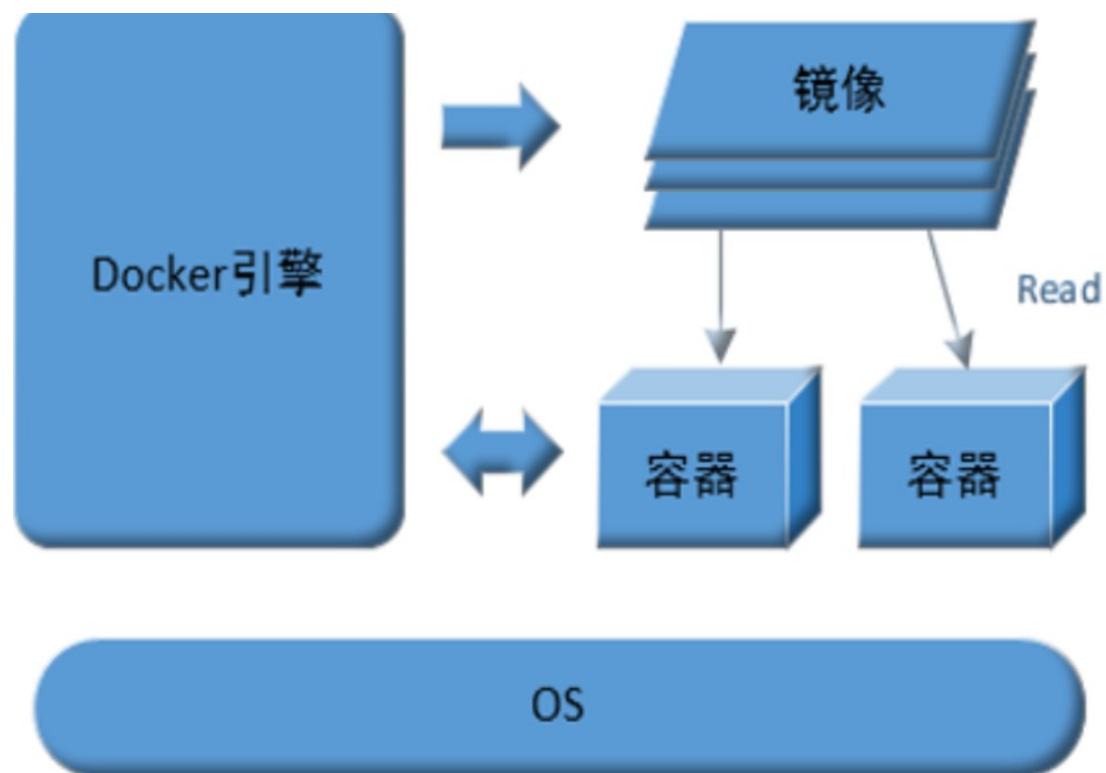
开发、运维、测试同事都觉得自己没问题，总结的时候会相互撕，影响产品上线效率，最重要的是还影响团队绩效，把你的钱蹭蹭的扣

打包 机制就不一样了，他很强，可以把我这个代码需要的一切，包括 tomcat, jdk, 甚至是 linux 文件都 打包到一起。打包这个东西叫做镜像



这个打出来的包叫做镜像。运行镜像的服务叫做容器。
可以发现 镜像可以很方便的做迁移，备份，存储，交付。

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。



这样，打出来的包，优点就出来了

1. 环境的强一致性。docker 镜像提供了除内核外完整的运行时环境，保证其他使用人员，能完整复现应用运行时环境。不会出现“这应用在 xxx 环境没问题”这种情况。
2. 迁移方便，持续交付和部署。一次打包，到处运行。

1、docker 是什么？

Docker 是一个开源项目，诞生于 2013 年初，最初是 dotCloud 公司内部的一个业余项目。它基于 Google 公司推出的 Go 语言实现。项目后来加入了 Linux 基金会，遵从了 Apache 2.0 协议，项目代码在 [GitHub](https://github.com) 上进行维护。

Docker 是一个开源的引擎，可以轻松的为任何应用创建一个轻量级的、可移植的、自给自足的容器。开发者可以打包他们的应用以及依赖包到一个可移植的镜像中，然后发布到任何支持 docker 的机器上运行。容器是完全使用沙箱机制，相互之间不会有任何接口调用。

Docker logo:

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。



Docker 的思想来自于集装箱，集装箱解决了什么问题？在一艘大船上，可以把货物规整的摆放起来。并且各种各样的货物被装在集装箱里，集装箱和集装箱之间不会互相影响。那么我就不用专门运送蔬菜的船和专门运送货物的船了。只要这些货物在集装箱里封装的好好的，那我就可以用一艘大船把他们都运走。

docker 就是类似的理念。云计算就好比大货轮。docker 就是集装箱。

2、docker 的优点

1) 快

运行时的性能快，管理操作(启动，停止，开始，重启等等)都是以秒或毫秒为单位的。

2) 敏捷

像虚拟机一样敏捷，而且会更便宜，在 bare metal(裸机)上布署像点个按钮一样简单。

3) 灵活

将应用和系统“容器化”，不添加额外的操作系统

4) 轻量

在一台服务器上可以布署 100~1000 个 Containers 容器。

5) 便宜

开源的，免费的，低成本的。

docker-ce: 社区版

docker-ee: 商业版

3、docker 缺点

所有容器共用 linux kernel 资源，资源能否实现最大限度利用，所以在安全上也会存在漏洞。

4、安装 Docker

主机 ip: 192.168.40.180

Centos7.6-centos7.9

4Gib/4vCPU

配置主机名:

```
[root@xianchaomaster1 ~]# hostnamectl set-hostname xianchaomaster1 && bash
```

关闭防火墙

```
[root@xianchaomaster1 ~]# systemctl stop firewalld && systemctl disable firewalld
```

关闭 iptables 防火墙

```
[root@xianchaomaster1 ~]# yum install iptables-services -y #安装 iptables
```

禁用 iptables

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

```
root@xianchaomaster1 ~]# service iptables stop  && systemctl disable iptables
```

清空防火墙规则

```
[root@xianchaomaster1 ~]# iptables -F
```

关闭 selinux

```
[root@xianchaomaster1 ~]# setenforce 0
```

```
[root@xianchaomaster1 ~]# sed -i 's/SELINUX=enforcing/SELINUX=disabled/g'
```

/etc/selinux/config

注意：修改 selinux 配置文件之后，重启机器，selinux 才能永久生效

```
[root@xianchaomaster1 ~]# getenforce
```

显示 Disabled 表示 selinux 关闭成功

#配置时间同步

```
[root@xianchaomaster1 ~]# yum install -y ntp ntpdate
```

```
[root@xianchaomaster1 ~]# ntpdate cn.pool.ntp.org
```

#编写计划任务

```
[root@xianchaomaster1 ~]# crontab -e
```

```
* */1 * * * /usr/sbin/ntpdate      cn.pool.ntp.org
```

重启 crond 服务使配置生效：

```
[root@xianchaomaster1 ~]# systemctl restart crond
```

安装基础软件包

```
[root@xianchaomaster1 ~]# yum install -y wget net-tools nfs-utils lrzsz gcc gcc-c++  
make cmake libxml2-devel openssl-devel curl curl-devel unzip sudo ntp libaio-devel wget vim  
ncurses-devel autoconf automake zlib-devel  python-devel epel-release openssh-server  
socat  ipvsadm conntrack
```

#安装 docker-ce

配置 docker-ce 国内 yum 源（阿里云）

```
[root@xianchaomaster1 ~]# yum-config-manager --add-repo
```

<http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo>

安装 docker 依赖包

```
[root@xianchaomaster1 ~]# yum install -y yum-utils device-mapper-persistent-data lvm2
```

安装 docker-ce

```
[root@xianchaomaster1 ~]# yum install docker-ce -y
```

#启动 docker 服务

```
[root@xianchaomaster1 ~]# systemctl start docker && systemctl enable docker
```

```
[root@xianchaomaster1 ~]# systemctl status docker
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

● docker.service - Docker Application Container Engine

Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; vendor preset: disabled)

Active: active (running) since Thu 2021-07-01 21:29:18 CST; 30s ago

Docs: <https://docs.docker.com>

看到 running, 表示 docker 正常运行

#查看 Docker 版本信息

```
[root@xianchaomaster1 ~]# docker version
```

5、开启包转发功能和修改内核参数

内核参数修改: br_netfilter 模块用于将桥接流量转发至 iptables 链, br_netfilter 内核参数需要开启转发。

```
[root@xianchaomaster1 ~]# modprobe br_netfilter
[root@xianchaomaster1 ~]# cat > /etc/sysctl.d/docker.conf <<EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
```

#使参数生效

```
[root@xianchaomaster1 ~]# sysctl -p /etc/sysctl.d/docker.conf
```

重启后模块失效, 下面是开机自动加载模块的脚本

在/etc/新建 rc.sysinit 文件

```
cat /etc/rc.sysinit
```

```
#!/bin/bash
for file in /etc/sysconfig/modules/*.modules ; do
[ -x $file ] && $file
done
```

在/etc/sysconfig/modules/目录下新建文件如下

```
cat /etc/sysconfig/modules/br_netfilter.modules
modprobe br_netfilter
```

增加权限

```
[root@xianchaomaster1 ~]# chmod 755 /etc/sysconfig/modules/br_netfilter.modules
```

重启机器模块也会自动加载

```
[root@localhost ~]# lsmod |grep br_netfilter
br_netfilter 22209 0
bridge 136173 1 br_netfilter
```

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追究。

注:

Docker 安装后出现: WARNING: bridge-nf-call-iptables is disabled 的解决办法:

```
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.bridge.bridge-nf-call-iptables = 1
```

```
net.ipv4.ip_forward = 1;
```

将 Linux 系统作为路由或者 VPN 服务就必须开启 IP 转发功能。当 linux 主机有多个网卡时一个网卡收到的信息是否能够传递给其他的网卡, 如果设置成 1 的话 可以进行数据包转发, 可以实现 VxLAN 等功能。不开启会导致 docker 部署应用无法访问。

```
#重启 docker
```

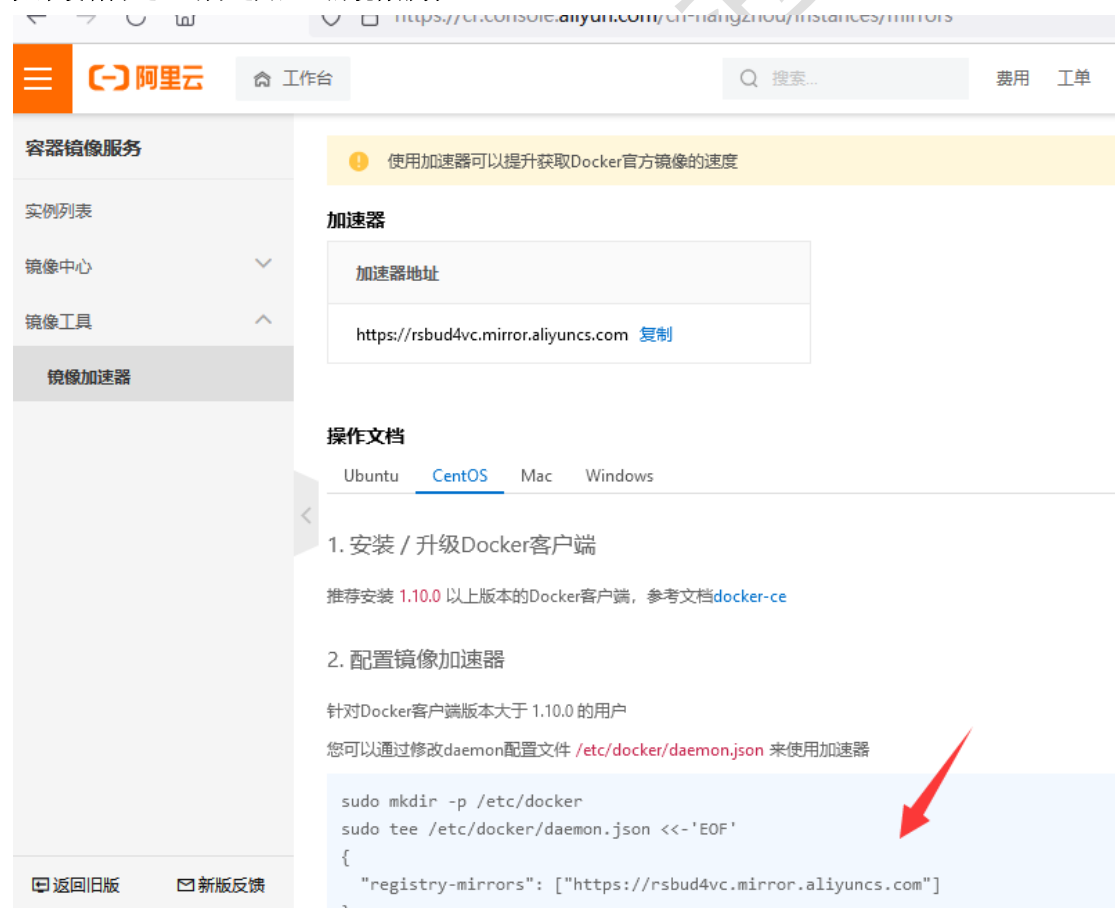
```
[root@xianchaomaster1 ~]# systemctl restart docker
```

6、配置 docker 镜像加速器

登陆阿里云镜像仓库

<https://cr.console.aliyun.com/cn-hangzhou/instances/mirrors>

如果没有开通, 可开通阿里云的镜像服务



容器镜像服务

实例列表

镜像中心

镜像工具

镜像加速器

使用加速器可以提升获取Docker官方镜像的速度

加速器

加速器地址

<https://rsbud4vc.mirror.aliyuncs.com> 复制

操作文档

Ubuntu CentOS Mac Windows

1. 安装 / 升级Docker客户端

推荐安装 1.10.0 以上版本的Docker客户端, 参考文档[docker-ce](#)

2. 配置镜像加速器

针对Docker客户端版本大于 1.10.0 的用户

您可以通过修改daemon配置文件 `/etc/docker/daemon.json` 来使用加速器

```
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json <<- 'EOF'
{
  "registry-mirrors": ["https://rsbud4vc.mirror.aliyuncs.com"]
}
```

找到镜像加速器, 然后按照箭头方向操作

修改/etc/docker/daemon.json, 变成如下

```
{
```

版权声明, 本文档全部内容版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

```
"registry-mirrors":["https://y8y6vosv.mirror.aliyuncs.com","https://registry.docker-
cn.com","https://docker.mirrors.ustc.edu.cn","https://dockerhub.azk8s.cn","http://hub-
mirror.c.163.com"]
}
```

让配置文件生效

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart docker
```

Docker Hub 镜像加速器列表

镜像加速器	镜像加速器地址	专属加速器?	其它加速?
Docker 中国官方 镜像	https://registry.docker-cn.com		Docker Hub
DaoCloud 镜像站	http://f1361db2.m.daocloud.io	可登录, 系统 分配	Docker Hub
Azure 中国镜像	https://dockerhub.azk8s.cn		Docker Hub、GCR、 Quay
科大镜像站	https://docker.mirrors.ustc.edu.cn		Docker Hub、GCR、 Quay
阿里云	<a href="https://<your_code>.mirror.aliyuncs.com">https://<your_code>.mirror.aliyuncs.com	需登录, 系统 分配	Docker Hub
七牛云	https://reg-mirror.qiniu.com		Docker Hub、GCR、 Quay
网易云	https://hub-mirror.c.163.com		Docker Hub
腾讯云	https://mirror.ccs.tencentyun.com		Docker Hub

7、docker 的基本用法

7.1 镜像相关操作

#从 dockerhub 查找镜像

```
[root@xianchaomaster1 ~]# docker search centos
```

NAME	DESCRIPTION	STARS
OFFICIAL AUTOMATED		
centos	The official build of CentOS.	6639
[OK]		
ansible/centos7-ansible	Ansible on Centos7	134
[OK]		
consol/centos-xfce-vnc	Centos container with "headless" VNC session...	129
[OK]		

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, **禁止私自传阅, 违者依法追责。**

[OK]

解释说明:

NAME: 镜像仓库源的名称

DESCRIPTION: 镜像的描述

OFFICIAL: 是否 docker 官方发布

stars: 类似 Github 里面的 star, 表示点赞、喜欢的意思。

AUTOMATED: 自动构建。

#下载镜像

```
[root@xianchaomaster1 ~]# docker pull centos
```

#查看本地镜像

```
[root@xianchaomaster1 ~]# docker images
```

#把镜像做成离线压缩包

```
[root@xianchaomaster1 ~]# docker save -o centos.tar.gz centos
```

#解压离线镜像包

```
[root@xianchaomaster1 ~]# docker load -i centos.tar.gz
```

#删除镜像

```
[root@xianchaomaster1 ~]# docker rmi -f centos:latest
```

7.2 容器相关操作

7.2.1 以交互式方式启动并进入容器

```
[root@xianchaomaster1 ~]# docker run --name=hello -it centos /bin/bash
```

```
[root@09c4933b5cd7 /]#
```

输入 exit, 退出容器, 退出之后容器也会停止, 不会再前台运行

#docker run 运行并创建容器

--name 容器的名字

-i 交互式

-t 分配伪终端

centos: 启动 docker 需要的镜像

/bin/bash 说明你的 shell 类型为 bash, bash shell 是最常用的一种 shell, 是大多数 Linux 发行版默认的 shell。 此外还有 C shell 等其它 shell。

7.2.2 以守护进程方式启动容器

```
[root@xianchaomaster1 ~]# docker run --name=hello1 -td centos
```

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, **禁止私自传阅, 违者依法追责。**

```
[root@xianchaomaster1 ~]# docker ps |grep hello1
1a2b73ba0ac2   centos          "/bin/bash"          hello1
```

-d 在后台运行 docker

```
[root@xianchaomaster1 ~]# docker exec -it hello1 /bin/bash
```

7.2.3 查看正在运行的容器

docker ps

```
[root@xianchaomaster1 ~]# docker ps -a #查看所有容器，包括运行和退出的容器
```

7.2.4 停止容器

```
docker stop hello1
```

7.2.5 启动已经停止的容器

```
docker start hello1
```

7.2.6 进入容器

```
docker exec -it hello1 /bin/bash
```

```
[root@xianchaomaster1 ~]# docker rm -f hello1 #删除容器
```

```
[root@xianchaomaster1 ~]# docker --help
```

#查看 docker 帮助命令

8、通过 docker 部署 nginx 服务

```
[root@xianchaomaster1 ~]# docker run --name nginx -p 80 -itd centos
```

#-p 把容器端口随机在物理机随机映射一个端口

```
[root@xianchaomaster1 ~]# docker ps | grep nginx
ecfa046e9681   centos          "/bin/bash"
5 seconds ago   Up 4 seconds    0.0.0.0:49153->80/tcp, :::49153->80/tcp   nginx
```

在 docker 里安装 nginx

```
docker exec -it nginx /bin/bash
```

```
[root@ecfa046e9681]# ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: tunl0@NONE: <NOARP> mtu 1480 qdisc noop state DOWN group default qlen 1000
    link/ipip 0.0.0.0 brd 0.0.0.0
21: eth0@if22: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
default
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

```
link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff link-netnsid 0
inet 172.17.0.3/16 brd 172.17.255.255 scope global eth0
    valid_lft forever preferred_lft forever
#通过上面可以看到容器的 ip 是 172.17.0.3
```

```
#yum 安装 nginx
rm -rf /etc/yum.repos.d/*
curl -o /etc/yum.repos.d/CentOS-Base.repo https://mirrors.aliyun.com/repo/Centos-vault-8.5.2111.repo
yum install wget -y
yum install nginx -y
```

```
安装文本编辑器 vim
yum install vim-enhanced -y
```

```
创建静态页面
mkdir /var/www/html -p
cd /var/www/html/
cat index.html
<html>
    <head>
        <title>nginx in docker</title>
    </head>
    <body>
        <h1>hello, My Name is xianchao</h1>
    </body>
</html>
```

```
修改 nginx 配置文件中的 root 路径, 如下
vim /etc/nginx/nginx.conf
root    /var/www/html/;
```

```
启动 nginx
/usr/sbin/nginx
```

访问 docker 里的 nginx 服务, 复制一个终端窗口, 执行如下命令

```
[root@xianchaomaster1 ~]# docker ps | grep nginx
ecfa046e9681    centos                                     "/bin/bash"
12 minutes ago    Up 12 minutes    0.0.0.0:49153->80/tcp, :::49153->80/tcp    nginx
```

#能查看到 nginx 容器在物理机映射的端口是 49153

```
[root@xianchaomaster1 ~]# curl http://192.168.40.180:49153
```

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追究。

```
<html>
  <head>
    <title>nginx in docker</title>
  </head>
  <body>
    <h1>hello, My Name is xianchao</h1>
  </body>
</html>
```

也可以直接访问容器的 ip:port

```
[root@xianchaomaster1 ~]# curl 172.17.0.3:80
```

```
<html>
  <head>
    <title>nginx in docker</title>
  </head>
  <body>
    <h1>hello, My Name is xianchao</h1>
  </body>
</html>
```

流量走向:

访问物理节点 ip:port (容器在物理节点映射的端口) → 容器 ip:port (容器里部署的服务的端口) → 就可以访问到容器里部署的应用了

containerd 容器概述

官方文档:

<https://containerd.io>

在 2016 年 12 月 14 日, Docker 公司宣布将 containerd 从 Docker 中分离, 由开源社区独立发展和运营。Containerd 完全可以单独运行并管理容器, 而 Containerd 的主要职责是镜像管理和容器执行。同时, Containerd 提供了 containerd-shim 接口封装层, 向下继续对接 runC 项目, 使得容器引擎 Docker Daemon 可以独立升级。

Containerd 可以在宿主机中管理完整的容器生命周期: 容器镜像的传输和存储、容器的执行和管理、存储和网络等。总结一下, 它主要负责干以下事情:

- 管理容器的生命周期(从创建容器到销毁容器)
- 拉取/推送容器镜像
- 存储管理(管理镜像及容器数据的存储)
- 调用 runC 运行容器(与 runC 等容器运行时交互)
- 管理容器网络接口及网络

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, **禁止私自传阅, 违者依法追责。**

ctr: containerd 的命令行客户端。

1、Containerd 和 Docker 之间的关系

Docker 包含 Containerd, Containerd 专注于运行时的容器管理, 而 Docker 除了容器管理之外, 还可以完成镜像构建之类的功能。

Containerd 提供的 API 偏底层, 不是给普通用户直接用的, 容器编排的开发者才需要 Containerd。

2、Containerd 在容器生态中扮演的角色

Containerd 并不是直接面向最终用户的, 而是主要用于集成到更上层的系统里, 比如 Kubernetes 等容器编排系统。

Containerd 以 daemon 的形式运行在系统上, 通过 unix domain socket 暴露底层的 grpc API, 上层系统可以通过这些 API 管理机器上的容器。

互动 1: K8S 为什么要放弃使用 Docker 作为容器运行时, 而使用 containerd 呢?

知道: 1

不: 2

Docker, Kubernetes 等工具来运行一个容器时会调用容器运行时 (CRI), 比如 containerd, CRI-O, 通过容器运行时来完成容器的创建、运行、销毁等实际工作, Docker 使用的是 containerd 作为其运行时; Kubernetes 支持 docker (在 k8s1.24 版本之前用, 1.24 开始废弃了)、containerd, CRI-O 等多种容器运行时, 这些容器运行时都遵循了 OCI 规范, 并通过 runc 来实现与操作系统内核交互来完成容器的创建和运行

备注: CRI

CRI 是一个插件接口, 它使 kubelet 能够使用各种容器运行时, 你需要在集群中的每个节点上都有一个可以正常工作的容器运行时, 这样 kubelet 能启动 Pod 及其容器。容器运行时接口 (CRI) 是 kubelet 和容器运行时之间通信的主要协议。

OCI:

OCI, Open Container Initiative, 是一个轻量级, 开放的治理结构 (项目), 在 Linux 基金会的支持下成立, 致力于围绕容器格式和运行时创建开放的行业标准。OCI 项目由 Docker, CoreOS (后来

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, **禁止私自传阅, 违者依法追责。**

被 Red Hat 收购了，相应的席位被 Red Hat 继承) 和容器行业中的其他领导者在 2015 年 6 月的时候启动。

如果你使用 Docker 作为 K8S 容器运行时的话，kubelet 需要先要通过 dockershim 去调用 Docker，再通过 Docker 去调用 containerd。

如果你使用 containerd 作为 K8S 容器运行时的话，kubelet 可以直接调用 containerd。

使用 containerd 不仅性能提高了（调用链变短了），而且资源占用也会变小（Docker 不是一个纯粹的容器运行时，具有大量其他功能）。

调用链

Docker 作为 k8s 容器运行时，调用关系如下：

kubelet --> docker shim (在 kubelet 进程中) --> dockerd --> containerd

Containerd 作为 k8s 容器运行时，调用关系如下：

kubelet --> cri plugin (在 containerd 进程中) --> containerd

3、containerd 安装和配置

安装 docker 会自动把 containerd 安装出来，也可以通过如下命令直接安装 containerd。

需要配置 docker-ce.repo 这个 yum 源：

```
[root@xianchaomaster1 ~]#yum install yum-utils -y
```

```
[root@xianchaomaster1 ~]#yum-config-manager --add-repo
```

```
http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
```

```
[root@xianchaomaster1 ~]# yum install containerd -y
```

```
[root@xianchaomaster1 ~]# systemctl enable containerd
```

```
[root@xianchaomaster1 ~]# systemctl start containerd
```

初始化 Containerd 配置：

```
[root@xianchaomaster1 ~]# containerd config default > /etc/containerd/config.toml
```

```
[root@xianchaomaster1 ~]# systemctl enable containerd
```

```
[root@xianchaomaster1 ~]# systemctl start containerd
```

替换 containerd 默认的 sand_box 镜像，编辑/etc/containerd/config.toml 文件：

```
sandbox_image = "k8s.gcr.io/pause:3.2" 替换成 registry.cn-hangzhou.aliyuncs.com/google_containers/pause-amd64:3.2
```

应用配置并重新运行 containerd 服务

```
[root@xianchaomaster1 ~]# systemctl daemon-reload
```

```
[root@xianchaomaster1 ~]# systemctl restart containerd
```

```
[root@xianchaomaster1 ~]# systemctl status containerd
```

```
containerd.service - containerd container runtime
```

```
Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; vendor preset: disabled)
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

Active: active (running) since Tue 2021-04-20 21:10:45 CST; 8s ago

```
[root@xianchaomaster1 ~]# ctr images pull registry.cn-  
hangzhou.aliyuncs.com/google_containers/pause-amd64:3.2
```

4、containerd 容器相关操作

#查看 containerd 命名空间

```
[root@xianchaomaster1 ~]# ctr namespace ls
```

ctr 有命名空间 namespace 来指定类似于工作空间的隔离区域。使用方法 `ctr -n default images ls` 来查看 default 命名空间的镜像，不加 `-n` 参数，默认也是使用 default 的命名空间。

#查看默认名称空间镜像有哪些

```
[root@xianchaomaster1 ~]# ctr image ls
```

#查看 k8s 命名空间下的镜像

```
[root@xianchaomaster1 ~]# ctr -n=k8s.io images ls
```

#拉取 busybox 镜像

```
[root@xianchaomaster1 ~]# ctr image pull docker.io/library/busybox:latest
```

注：必须全路径，从 dockerhub 上下载默认 busybox 镜像。

```
[root@xianchaomaster1 ~]# ctr images rm 删除镜像
```

#压缩镜像

```
[root@xianchaomaster1 ~]# ctr images pull docker.io/library/mysql:latest
```

```
[root@xianchaomaster1 ~]# ctr images export mysql.tar.gz
```

docker.io/library/mysql:latest

#导出镜像

```
[root@xianchaomaster1 ~]# ctr images import mysql.tar.gz
```

互动 1: docker 的镜像和 containerd 镜像通用吗?

答：通用的，`docker save -o` 生成的镜像文件，可以基于 `ctr images import` 导出来

9、dockerfile 语法详解

Dockerfile 是一个用来构建镜像的文本文件，文本内容包含了一条条构建镜像所需的指令和说明。

基于 Dockerfile 构建镜像可以使用 `docker build` 命令。`docker build` 命令中使用 `-f` 可以指定具体的 dockerfile 文件

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，**禁止私自传阅，违者依法追责。**

```
FROM centos
MAINTAINER xianchao
RUN rm -rf /etc/yum.repos.d/*
COPY Centos-vault-8.5.2111.repo /etc/yum.repos.d/
RUN yum install wget -y
RUN yum install nginx -y
COPY index.html /usr/share/nginx/html/
EXPOSE 80
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
```

#创建 index.html

```
vim index.html
<html>
<head>
    <title>page added to dockerfile</title>
</head>
<body>
    <h1>i am in df_test </h1>
</body>
</html>
```

dockerfile 构建过程:

从基础镜像运行一个容器

执行一条指令, 对容器做出修改

执行类似 docker commit 的操作, 提交一个新的镜像层

再基于刚提交的镜像运行一个新的容器

执行 dockerfile 中的下一条指令, 直至所有指令执行完毕

(1) FROM

基础镜像, 必须是可以下载下来的, 定制的镜像都是基于 FROM 的镜像, 这里的 centos 就是定制需要的基础镜像。后续的操作都是基于 centos 镜像。

(2) MAINTAINER

指定镜像的作者信息

(3) RUN: 指定在当前镜像构建过程中要运行的命令

包含两种模式

1、Shell

RUN <command> (shell 模式, 这个是最常用的, 需要记住)

RUN echo hello

2、exec 模式

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

`RUN [ "executable", "param1", "param2" ] (exec 模式)`

`RUN [ "/bin/bash", "-c", "echo hello" ]`

等价于 `/bin/bash -c echo hello`

(4) EXPOSE 指令

仅仅只是声明端口。

作用：

- 1、帮助镜像使用者理解这个镜像服务的守护端口，以方便配置映射。
- 2、在运行时使用随机端口映射时，也就是 `docker run -P` 时，会自动随机映射 EXPOSE 的端口。
- 3、可以是一个或者多个端口，也可以指定多个 EXPOSE

格式：EXPOSE <端口 1> [<端口 2>...]

(5) CMD

类似于 RUN 指令，用于运行程序，但二者运行的时间点不同：

- 1、CMD 在 `docker run` 时运行。
- 2、RUN 是在 `docker build` 构建镜像时运行的

作用：为启动的容器指定默认要运行的程序，程序运行结束，容器也就结束。CMD 指令指定的程序可被 `docker run` 命令行参数中指定要运行的程序所覆盖。

`CMD [ "executable", "param1", "param2" ] (exec 模式)`

`CMD command` (shell 模式)

`CMD [ "param1", "param2" ]` (作为 ENTRYPOINT 指令的默认参数)

例：`cd /root/dockerfile/test`

`cat dockerfile`

`#first dockerfile`

`FROM centos`

`MAINTAINER xianchao`

`RUN yum clean all`

`RUN yum install nginx -y`

`EXPOSE 80`

`CMD [ "/usr/sbin/nginx", "-g", "daemon off;" ]`

构建镜像：

`docker build -t="dockerfile/test-cmd:v1" .`

基于上面构建的镜像运行一个容器

`docker run -p 80 --name cmd_test2 -d dockerfile/test-cmd:v1`

(不需要跟 `nginx -g "daemon off;"` 了)

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

docker ps 可以看到下面信息

```
b903d5a71279      dockerfile/test-cmd:v1      "/usr/sbin/nginx -
g ..." 7 seconds ago      Up 6 seconds      0.0.0.0:32770->80/tcp      cmd_test2
```

(6) ENTRYPOINT

类似于 CMD 指令，但其不会被 docker run 的命令行参数指定的指令所覆盖，而且这些命令行参数会被当作参数送给 ENTRYPOINT 指令指定的程序。

但是，如果运行 docker run 时使用了 --entrypoint 选项，将覆盖 entrypoint 指令指定的程序。

优点：在执行 docker run 的时候可以指定 ENTRYPOINT 运行所需的参数。

注意：如果 Dockerfile 中如果存在多个 ENTRYPOINT 指令，仅最后一个生效。

格式：

ENTRYPOINT ["executable" , "param1" , "param2"] (exec 模式)

ENTRYPOINT command (shell 模式)

可以搭配 CMD 命令使用：一般是变参才会使用 CMD，这里的 CMD 等于是在给 ENTRYPOINT 传参，以下示例会提到。

示例：

假设已通过 Dockerfile 构建了 nginx:test 镜像：

```
FROM nginx
```

```
ENTRYPOINT ["nginx", "-c"] # 定参
```

```
CMD ["/etc/nginx/nginx.conf"] # 变参
```

1、不传参运行

```
$ docker run nginx:test
```

容器内会默认运行以下命令，启动主进程。

```
nginx -c /etc/nginx/nginx.conf
```

2、传参运行

```
$ docker run nginx:test -c /etc/nginx/new.conf
```

容器内会默认运行以下命令，启动主进程 (/etc/nginx/new.conf: 假设容器内已有此文件)

```
nginx -c /etc/nginx/new.conf
```

(7) COPY

COPY<src>..<dest>

COPY["<src>" ... "<dest>"]

复制指令，从上下文目录中复制文件或者目录到容器里指定路径。

格式：

COPY [--chown=<user>:<group>] <源路径 1>... <目标路径>

COPY [--chown=<user>:<group>] ["<源路径 1>" , ... "<目标路径>"]

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

[--chown=<user>:<group>]: 可选参数, 用户改变复制到容器内文件的拥有者和属组。

<源路径>: 源文件或者源目录, 这里可以是通配符表达式, 其通配符规则要满足 Go 的 filepath.Match 规则。例如:

```
COPY hom* /mydir/
COPY hom?.txt /mydir/
```

<目标路径>: 容器内的指定路径, 该路径不用事先建好, 路径不存在的话, 会自动创建。

(8) ADD

```
ADD <src>...<dest>
ADD [ "<src>" ... "<dest>" ]
```

ADD 指令和 COPY 的使用格式一致 (同样需求下, 官方推荐使用 COPY)。功能也类似, 不同之处如下:

ADD 的优点: 在执行 <源文件> 为 tar 压缩文件的话, 压缩格式为 gzip, bzip2 以及 xz 的情况下, 会自动复制并解压到 <目标路径>。

ADD 的缺点: 在不解压的前提下, 无法复制 tar 压缩文件。会令镜像构建缓存失效, 从而可能会令镜像构建变得比较缓慢。具体是否使用, 可以根据是否需要自动解压来决定。

ADD vs COPY

ADD 包含类似 tar 的解压功能

如果单纯复制文件, dockerfile 推荐使用 COPY

```
例; 替换/usr/share/nginx 下的 index.html
cd /root/dockerfile/test1
cat dockerfile
FROM centos
MAINTAINER xianchao
RUN yum install wget -y
RUN yum install nginx -y
COPY index.html /usr/share/nginx/html/
EXPOSE 80
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
```

```
vim index.html
<html>
<head>
```

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

```
<title>page added to dockerfile</title>
</head>
<body>
    <h1>i am in df_test </h1>
</body>
</html>
```

```
docker build -t="dockerfile/copy:v1" .
```

```
docker run -d -p 80 --name html3 dockerfile/copy:v1
```

```
docker ps | grep html3
```

显示如下:

```
478868402ac4      dockerfile/copy:v1      "/usr/sbin/nginx -
g ..." 15 seconds ago      Up 12 seconds      0.0.0.0:32771->80/tcp      html3
```

```
curl 192.168.40.180:32771
```

显示的就是替换后的页面

```
<html>
<head>
    <title>page added to dockerfile</title>
</head>
<body>
    <h1>i am in df_test </h1>
</body>
</html>
```

(9) VOLUME

定义匿名数据卷。在启动容器时忘记挂载数据卷，会自动挂载到匿名卷。

作用:

- 1、避免重要的数据，因容器重启而丢失，这是非常致命的。
- 2、避免容器不断变大。

格式:

```
VOLUME [ "<路径 1>", "<路径 2>" ... ]
```

```
VOLUME <路径>
```

在启动容器 `docker run` 的时候，我们可以通过 `-v` 参数修改挂载点。

```
VOLUME [ "/data" ]
```

(10) WORKDIR

指定工作目录。用 `WORKDIR` 指定的工作目录，会在构建镜像的每一层中都存在。（`WORKDIR` 指定的工作目录，必须是提前创建好的）。

`docker build` 构建镜像过程中的，每一个 `RUN` 命令都是新建的一层。只有通过 `WORKDIR` 创建的目录才

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

会一直存在。

格式:

WORKDIR <工作目录路径>

WORKDIR /path/to/workdir

(填写绝对路径)

(11) ENV

设置环境变量

ENV <key> <value>

ENV <key>=<value>...

以下示例设置 `NODE_VERSION=6.6.6`，在后续的指令中可以通过 `$NODE_VERSION` 引用:

ENV NODE_VERSION 6.6.6

```
RUN curl -sLO "https://nodejs.org/dist/v$NODE_VERSION/node-v$NODE_VERSION-linux-x64.tar.xz" \
  && curl -sLO "https://nodejs.org/dist/v$NODE_VERSION/SUMS256.txt.asc"
```

(12) USER

用于指定执行后续命令的用户和用户组，这边只是切换后续命令执行的用户（用户和用户组必须提前已经存在）。

格式:

USER <用户名>[:<用户组>]

USER daemon

USER nginx

USER user

USER uid

USER user:group

USER uid:gid

USER user:gid

USER uid:group

(13) ONBUILD

用于延迟构建命令的执行。简单的说，就是 Dockerfile 里用 ONBUILD 指定的命令，在本次构建镜像的过程中不会执行（假设镜像为 `test-build`）。当有新的 Dockerfile 使用了之前构建的镜像 `FROM test-build`，这时执行新镜像的 Dockerfile 构建时候，会执行 `test-build` 的 Dockerfile 里的 ONBUILD 指定的命令。

格式:

ONBUILD <其它指令>

为镜像添加触发器

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

当一个镜像被其他镜像作为基础镜像时需要写上 ONBUILD
会在构建时插入触发器指令

例：演示 ONBUILD 指令

```
cat index.html
<html>
<head>
    <title>page added to dockerfile</title>
</head>
<body>
    <h1>i am in df_test </h1>
</body>
</html>
```

```
vim dockerfile
FROM centos
MAINTAINER xianchao
RUN yum install wget -y
RUN yum install nginx -y
ONBUILD COPY index.html /usr/share/nginx/html/
EXPOSE 80
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
```

```
docker build -t="onbuild-nginx:v1" .
docker run -d --name html4 -p 80 onbuild-nginx:v1
docker ps | grep html4
```

显示如下：

```
65f4a5be9355      onbuild-nginx:v1      "/usr/sbin/nginx -
g ..." 14 seconds ago    Up 11 seconds        0.0.0.0:32772->80/tcp    html4
```

```
curl 192.168.40.180:32772
```

显示还是以前 nginx 默认的内容，没有被替换，表示 ONBUILD 这个指令后面的 COPY 没有生效

还是在刚才在路径下构建新的镜像

```
vim dockerfile
FROM onbuild-nginx:v1
MAINTAINER xianchao
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
EXPOSE 80
```

```
docker build -t="onbuild-nginx1" .
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

```
docker run -d --name htm5 -p 80 onbuild-nginx1
```

```
docker ps | grep htm5
```

显示如下：

```
e56542310692      onbuild-nginx1      "/usr/sbin/nginx -
g ..." 12 seconds ago      Up 8 seconds      0.0.0.0:32773->80/tcp      htm5
```

```
curl 192.168.40.180:32773
```

显示如下：

```
<html>
<head>
    <title>page added to dockerfile</title>
</head>
<body>
    <h1>i am in df_test </h1>
</body>
</html>
```

显示的就是已经重新构建的镜像，页面就是替换之后的了，说明我们基于 ONBUILD 指令的镜像作为基础镜像，在构建镜像，会触发 ONBUILD 后面的 COPY 命令运行

(14) LABEL

LABEL 指令用来给镜像添加一些元数据 (metadata)，以键值对的形式，语法格式如下：

```
LABEL <key>=<value> <key>=<value> <key>=<value> ...
```

比如我们可以添加镜像的作者：

```
LABEL org.opencontainers.image.authors="xianchao"
```

(15) HEALTHCHECK

用于指定某个程序或者指令来监控 docker 容器服务的运行状态。

格式：

HEALTHCHECK [选项] CMD <命令>：设置检查容器健康状况的命令

HEALTHCHECK NONE：如果基础镜像有健康检查指令，使用这行可以屏蔽掉其健康检查指令

HEALTHCHECK [选项] CMD <命令>：这边 CMD 后面跟随的命令使用，可以参考 CMD 的用法。

(16) ARG

构建参数，与 ENV 作用一至。不过作用域不一样。ARG 设置的环境变量仅对 Dockerfile 内有效，也就是说只有 docker build 的过程中有效，构建好的镜像内不存在此环境变量。

构建命令 docker build 中可以用 --build-arg <参数名>=<值> 来覆盖。

格式：

```
ARG <参数名>[=<默认值>]
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

10、dockerfile 构建 nginx 镜像

```
mkdir dockerfile
```

```
cd dockerfile/
```

```
vim dockerfile
```

```
FROM centos
MAINTAINER xianchao
RUN rm -rf /etc/yum.repos.d/*
COPY Centos-vault-8.5.2111.repo /etc/yum.repos.d/
RUN yum install wget -y
RUN yum install nginx -y
COPY index.html /usr/share/nginx/html/
EXPOSE 80
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
```

```
vim index.html
```

```
<html>
<head>
    <title>page added to dockerfile</title>
</head>
<body>
    <h1>Hello, My Name is Xianchao,My wechat is luckylucky421302 </h1>
</body>
</html>
```

构建镜像:

```
docker build -t="xianchao/nginx:v1" .
```

查看镜像是否构建成功:

```
docker images | grep nginx
```

显示如下说明镜像部署成功:

```
dockerfile/nginx latest bae97a76499 About a minute ago 344MB
```

基于刚才的镜像启动容器:

```
docker run -d -p 80 --name html2 xianchao/nginx:v1
```

查看容器具体信息可按如下命令:

```
[root@xianchaomaster1 dockerfile]# docker ps | grep html
```

```
bdbel40d5dc9 xianchao/nginx:v1 "/usr/sbin/nginx -g ..."
17 seconds ago Up 15 seconds 0.0.0.0:49154->80/tcp, :::49154->80/tcp html2
```

查看容器里部署的 nginx 网站的内容:

```
curl http://192.168.40.180:49154
```

显示如下:

```
<html>
<head>
    <title>page added to dockerfile</title>
</head>
```

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

```
<body>
    <h1>Hello, My Name is Xianchao,My wechat is luckylucky421302 </h1>
</body>
</html>
```

注意：ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]

表示容器运行时，自动启动容器里的 nginx 服务

11、dockerfile 构建 tomcat 镜像

下面需要的压缩包在下面需要的压缩包在课件

```
mkdir tomcat8
cd tomcat8
把 apache-tomcat-8.0.26.tar.gz 和 jdk-8u45-linux-x64.rpm 传到这个目录下
ls 可以查看到 tomcat8 下面有如下几个文件
[root@localhost tomcat8]# ls
apache-tomcat-8.0.26.tar.gz  dockerfile  jdk-8u45-linux-x64.rpm
cat dockerfile
FROM centos
MAINTAINER xianchao
RUN rm -rf /etc/yum.repos.d/*
COPY Centos-vault-8.5.2111.repo /etc/yum.repos.d/
RUN yum install wget -y
ADD jdk-8u45-linux-x64.rpm /usr/local/
ADD apache-tomcat-8.0.26.tar.gz /usr/local/
RUN cd /usr/local && rpm -ivh jdk-8u45-linux-x64.rpm
RUN mv /usr/local/apache-tomcat-8.0.26 /usr/local/tomcat8
EXPOSE 8080
```

开始构建镜像

```
docker build -t="tomcat8:v1" .
```

运行一个容器

```
docker run --name tomcat8 -itd -p 8080 tomcat8:v1
```

进入到容器

```
docker exec -it tomcat8 /bin/bash
```

启动 tomcat

```
/usr/local/tomcat8/bin/startup.sh
```

查看进程，看看是否启动成功

```
ps -ef | grep tomcat
```

打开新的终端窗口，查看刚才创建的 tomcat8 这个容器的详细信息：

```
docker ps | grep tomcat
```

显示如下信息

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

```
4d4c91cff4b5      tomcat8:v1      "/bin/bash"
About a minute ago   Up About a minute   0.0.0.0:32776->8080/tcp   tomcat8
```

通过上面可以看到，tomcat 在宿主机上映射的端口是 32776

这样我们请求 docker 节点的 ip:32776，就可以访问到 tomcat 的内容了

通过这些步骤可以实现通过 dockerfile 构建 tomcat 镜像了

刚才我们在构建 tomcat 镜像时候，基于镜像运行容器，但是需要进入到容器，手动启动 tomcat 服务，那如果想要启动容器，tomcat 也自动起来，需要按照如下方法构建镜像：

```
FROM centos
MAINTAINER xianchao
RUN rm -rf /etc/yum.repos.d/*
COPY Centos-vault-8.5.2111.repo /etc/yum.repos.d/
RUN yum install wget -y
ADD jdk-8u45-linux-x64.rpm /usr/local/
ADD apache-tomcat-8.0.26.tar.gz /usr/local/
RUN cd /usr/local && rpm -ivh jdk-8u45-linux-x64.rpm
RUN mv /usr/local/apache-tomcat-8.0.26 /usr/local/tomcat8
ENTRYPOINT /usr/local/tomcat8/bin/startup.sh && tail -F
/usr/local/tomcat8/logs/catalina.out
EXPOSE 8080
```

把 Go 代码基于 dockerfile 做成镜像

1、linux 机器安装 go（我用的是 centos7.9 操作系统）

```
[root@xianchaomaster1 ~]# yum install go -y
```

2、创建源码文件

生产环境代码是开发人员写的，运维不用写

```
[root@xianchaomaster1 ~]# mkdir test
```

```
[root@xianchaomaster1 ~]# cd test/
```

```
[root@xianchaomaster1 test]# cat main.go
```

```
package main
```

```
import (
```

```
    "net/http"
```

```
    "github.com/gin-gonic/gin"
```

```
)
```

```
func statusOKHandler(c *gin.Context) {
```

```
    c.JSON(http.StatusOK, gin.H{"status": "success~welcome to study"})
```

```
}
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

```
func versionHandler(c *gin.Context) {  
    c.JSON(http.StatusOK, gin.H{"version": "v1.1 版本"})  
}
```

```
func main() {  
    router := gin.New()  
    router.Use(gin.Recovery())  
    router.GET("/", statusOKHandler)  
    router.GET("/version", versionHandler)  
    router.Run(":8080")  
}
```

3、Go mod 初始化项目

```
[root@xianchaomaster1 test]# go mod init test  
#设置代理  
[root@xianchaomaster1 test]# go env -w GOPROXY=https://goproxy.cn,direct  
[root@xianchaomaster1 test]# go mod tidy  
#构建源码  
[root@xianchaomaster1 test]# CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o k8s-demo  
main.go
```

4、编写 dockerfile 文件

```
[root@xianchaomaster1 test]# cat Dockerfile  
FROM alpine  
WORKDIR /data/app  
ADD k8s-demo /data/app/  
CMD ["/bin/sh", "-c", "./k8s-demo"]
```

5、构建镜像

```
[root@xianchaomaster1 test]# docker build -t xianchao/k8sdemo:v1 .
```

6、运行镜像

```
[root@xianchaomaster1 test]# docker run -d --name go -p 30180:8080 xianchao/k8sdemo:v1
```

7、访问 go 容器

可以在根物理机同网段的其他机器或者浏览器访问 xianchaomaster1 机器的 ip:30180，就可以把请求代理到 go 容器了

12、Docker 容器的数据管理

Docker 容器的数据卷

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

什么是数据卷？

数据卷是经过特殊设计的目录，可以绕过联合文件系统（UFS），为一个或者多个容器提供访问，数据卷设计的目的，在于数据的永久存储，它完全独立于容器的生存周期，因此，docker 不会在容器删除时删除其挂载的数据卷，也不会存在类似的垃圾收集机制，对容器引用的数据卷进行处理，同一个数据卷可以只支持多个容器的访问。

数据卷的特点：

1. 数据卷在容器启动时初始化，如果容器使用的镜像在挂载点包含了数据，这些数据会被拷贝到新初始化的数据卷中
2. 数据卷可以在容器之间共享和重用
3. 可以对数据卷里的内容直接进行修改
4. 数据卷的变化不会影像镜像的更新
5. 卷会一直存在，即使挂载数据卷的容器已经被删除

数据卷的使用：

1. 为容器添加数据卷

```
docker run -v /datavolume:/data -it centos /bin/bash
```

如：

```
docker run --name volume -v ~/datavolume:/data -itd centos /bin/bash
```

注：~/datavolume 为宿主机目录，/data 为 docker 启动的 volume 容器的里的目录
这样在宿主机的/datavolume 目录下创建的数据就会同步到容器的/data 目录下

（1）为数据卷添加访问权限

```
docker run --name volumel -v ~/datavolumel:/data:ro -itd centos /bin/bash
```

添加只读权限之后在 docker 容器的/data 目录下就不能在创建文件了，为只读权限；在宿主机下的 /datavolumel 下可以创建东西

2. 使用 dockerfile 构建包含数据卷的镜像

dockerfile 指令：

```
volume[ "/data" ]
```

```
cat dockerfile
```

```
FROM centos
```

```
VOLUME ["/datavolume3", "/datavolume6"]
```

```
CMD /bin/bash
```

使用如下构建镜像

```
docker build -t="volume" .
```

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

启动容器

```
docker run --name volume-dubble -it volume
```

会看到这个容器下有两个目录，/datavolume3 和 /datavolume6

13、Docker 的数据卷容器

什么是数据卷容器：

命名的容器挂载数据卷，其他容器通过挂载这个容器实现数据共享，挂载数据卷的容器，就叫做数据卷容器

挂载数据卷容器的方法

```
docker run --volumes-from [container name]
```

例：

```
docker run --name data-volume -itd volume (volume 这个镜像是上面创建的带两个数据卷 /datavolume3 和 /datavolume6 的镜像)
```

```
docker exec -it data-volume /bin/bash (进入到容器中)
```

```
touch /datavolume6/lucky.txt
```

退出容器 exit

创建一个新容器挂载刚才 data-volume 这个容器创建的数据卷

```
docker run --name data-volume2 --volumes-from data-volume -itd centos /bin/bash
```

进入到新创建的容器

```
docker exec -it data-volume2 /bin/bash
```

查看容器的 /datavolume6 目录下是否新创建了 lucky.txt 文件

```
cd /datavolume6
```

可以看见有刚才在上一个容器创建的文件 lucky.txt

14、docker 数据卷的备份和还原

数据备份方法：

```
docker run --volumes-from [container name] -v $(pwd):/backup centos tar czvf /backup/backup.tar [container data volume]
```

例子：

```
docker run --volumes-from data-volume2 -v /root/backup:/backup --name datavolume-copy centos tar zcvf /backup/data-volume2.tar.gz /datavolume6
```

数据还原方法：

```
docker run --volumes-from [container name] -v $(pwd):/backup centos tar xzvf /backup/backup.tar.gz [container data volume]
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

例:

```
docker exec -it data-volume2 /bin/bash
cd /datavolume6
rm -rf lucky.txt
```

```
docker run --volumes-from data-volume2 -v /root/backup:/backup centos tar zxvf
/backup/data-volume2.tar.gz -C /datavolume6
```

```
docker exec -it data-volume2 /bin/bash
cd /datavolume6
可以看到还原后的数据
```

15、docker 容器互联

docker run 创建 Docker 容器时, 可以用--net 选项指定容器的网络模式, Docker 有以下 4 种网络模式:

- bridge 模式: 使--net =bridge 指定, 默认设置;
- host 模式: 使--net =host 指定;
- none 模式: 使--net =none 指定;
- container 模式: 使用--net =container:NAME or ID 指定。

15.1 docker 容器的网络基础

1、docker0:

安装 docker 的时候, 会生成一个 docker0 的虚拟网桥

2、Linux 虚拟网桥的特点:

可以设置 ip 地址

相当于拥有一个隐藏的虚拟网卡

```
docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
link/ether 02:42:28:ae:c0:42 brd ff:ff:ff:ff:ff:ff
inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
    valid_lft forever preferred_lft forever
inet6 fe80::42:28ff:feae:c042/64 scope link
```

每运行一个 docker 容器都会生成一个 veth 设备对, 这个 veth 一个接口在容器里, 一个接口在物理机上。

3、安装网桥管理工具:

```
yum install bridge-utils -y
```

brctl show 可以查看到一个 docker0 的网桥设备, 下面有很多接口, 每个接口都表示一个启动的

版权声明, 本文档全部内容版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

docker 容器，因为我在 docker 上启动了很多容器，所以 interfaces 较多，如下所示：

```
[root@localhost ~]# brctl show
bridge name      bridge id        STP enabled      interfaces
docker0          8000.023cad42f31 no                veth0d6a311
                  veth281607e
                  veth3e35c35
                  veth422bfcc
                  veth4fc6204
                  veth6bc7c79
                  veth8aa8048
                  veth95c98be
                  vethb4dd450
                  vethc250aa0
                  vetheea2342
                  vethf52e44d
                  vethf865a99
                  vethfc4dlbf
```

15.2 docker 容器的互联

下面用到的镜像的 dockerfile 文件如下：

```
cd dockerfile/inter-image
vim dockerfile
FROM centos
RUN yum install wget -y
RUN yum install nginx -y
RUN sed -i "7s/^/#/g" /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD /bin/bash
```

```
[root@xianchaomaster1]# docker build -t="inter-image" .
```

允许所有容器间互联（也就是访问）

第一种方法：

例：

（1）基于上面的 inter-image 镜像启动第一个容器 test1

```
docker run --name test1 -itd inter-image
```

进入到容器里面启动 nginx：

```
/usr/sbin/nginx
```

（2）基于上面的 inter-image 镜像启动第二个容器 test2

```
docker run --name test2 -itd inter-image
```

（3）进入到 test1 容器和 test2 容器，可以看两个容器的 ip，分别是 172.17.0.20 和 172.17.0.21

```
docker exec -it test2 /bin/bash
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

ping 172.17.0.20 可以看见能 ping 同 test1 容器的 ip

curl http://172.17.0.20

可以访问到 test1 容器的内容

上述方法假如 test1 容器重启，那么在启动就会重新分配 ip 地址，所以为了使 ip 地址变了也可以访问可以采用下面的方法

15.2.1 docker link 设置网络别名

第二种方法：

可以给容器起一个代号，这样可以直接以代号访问，避免了容器重启 ip 变化带来的问题

--link

docker run --link=[CONTAINER_NAME]:[ALIAS] [IMAGE][COMMAND]

例：

1. 启动一个 test3 容器

docker run --name test3 -itd inter-image /bin/bash

2. 启动一个 test5 容器，--link 做链接，那么当我们重新启动 test3 容器时，就算 ip 变了，也没关系，我们可以在 test5 上 ping 别名 webtest

docker run --name test5 -itd --link=test3:webtest inter-image /bin/bash

3. test3 和 test5 的 ip 分别是 172.17.0.22 和 172.17.0.24

4. 重启 test3 容器

docker restart test3

发现 ip 变成了 172.17.0.25

5. 进入到 test5 容器

docker exec -it test5 /bin/bash

ping test3 容器的 ip 别名 webtest 可以 ping 通，尽管 test3 容器的 ip 变了也可以通

16、docker 容器的网络模式

docker run 创建 docker 容器时，可以用 --net 选项指定容器的网络模式，Docker 有以下 4 种网络模式：

bridge 模式：使 --net =bridge 指定，默认设置；

host 模式：使 --net =host 指定；

none 模式：使 --net =none 指定；

container 模式：使用 --net =container:NAME or ID 指定。

16.1 none 模式

Docker 网络 none 模式是指创建的容器没有网络地址，只有 lo 网卡

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

```
[root@xianchaomaster1 ~]# docker run -itd --name none --net=none --privileged=true centos
[root@xianchaomaster1 ~]# docker exec -it none /bin/bash
```

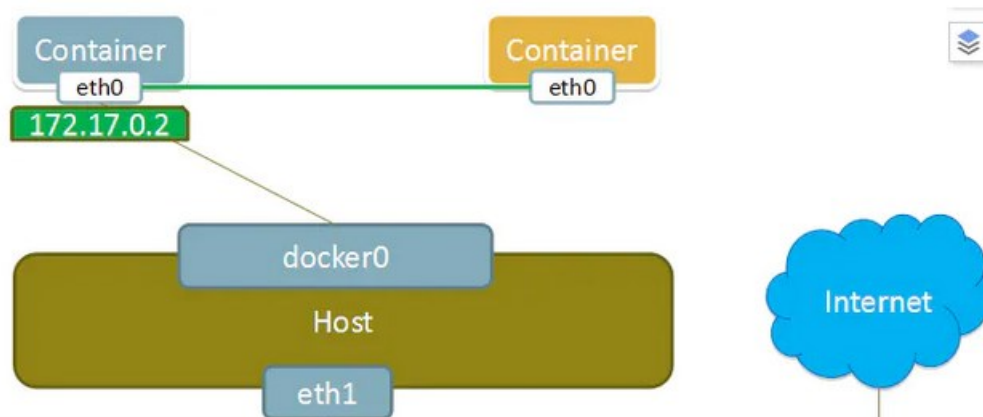
```
[root@05dbf3f2daaf /]# ip addr
```

#只有本地 lo 地址

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
```

16.2 container 模式

Docker 网络 container 模式是指，创建新容器的时候，通过--net container 参数，指定其和已经存在的某个容器共享一个 Network Namespace。如下图所示，右方黄色新创建的 container，其网卡共享左边容器。因此就不会拥有自己独立的 IP，而是共享左边容器的 IP 172.17.0.2，端口范围等网络资源，两个容器的进程通过 lo 网卡设备通信。



#和已经存在的 none 容器共享网络

```
[root@xianchaomaster1 ~]# docker run --name container2 --net=container:none -it --privileged=true centos
```

```
[root@05dbf3f2daaf /]# ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
```

16.3 bridge 模式

默认选择 bridge 的情况下，容器启动后会通过 DHCP 获取一个地址

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

```
#创建桥接网络
[root@xianchaomaster1~]# docker run --name bridge -it --privileged=true centos bash

[root@a131580fb605 /]# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen
1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
64: eth0@if65: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group
default
    link/ether 02:42:ac:11:00:0d brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.17.0.13/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
```

16.4 host 模式

Docker 网络 host 模式是指共享宿主机的网络

```
#共享宿主机网络
[root@xianchaomaster1~]# docker run --name host -it --net=host --privileged=true centos
bash
```

17、docker 资源配额

17.1 docker 容器控制 cpu

Docker 通过 cgroup 来控制容器使用的资源限制，可以对 docker 限制的资源包括 CPU、内存、磁盘

17.1.1 指定 docker 容器可以使用的 cpu 份额

#查看配置份额的帮助命令：

```
[root@xianchaomaster1 ~]# docker run --help | grep cpu-shares
cpu 配额参数: -c, --cpu-shares int
```

CPU shares (relative weight) 在创建容器时指定容器所使用的 CPU 份额值。cpu-shares 的值不能保证可以获得 1 个 vcpu 或者多少 GHz 的 CPU 资源，仅仅只是一个弹性的加权值。

默认每个 docker 容器的 cpu 份额值都是 1024。在同一个 CPU 核心上，同时运行多个容器时，容器的 cpu 加权的效果才能体现出来。

例：两个容器 A、B 的 cpu 份额分别为 1000 和 500，结果会怎么样？

情况 1：A 和 B 正常运行，占用同一个 CPU，在 cpu 进行时间片分配的时候，容器 A 比容器 B 多一倍的机会获得 CPU 的时间片。

情况 2：分配的结果取决于当时其他容器的运行状态。比如容器 A 的进程一直是空闲的，那么容器 B

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

是可以获取比容器 A 更多的 CPU 时间片的；比如主机上只运行了一个容器，即使它的 cpu 份额只有 50，它也可以独占整个主机的 cpu 资源。

cgroups 只在多个容器同时争抢同一个 cpu 资源时，cpu 配额才会生效。因此，无法单纯根据某个容器的 cpu 份额来确定有多少 cpu 资源分配给它，资源分配结果取决于同时运行的其他容器的 cpu 分配和容器中进程运行情况。

例 1：给容器实例分配 512 权重的 cpu 使用份额

参数： `--cpu-shares 512`

```
[root@xianchaomaster1 ~]# docker run -it --cpu-shares 512 centos /bin/bash
```

```
[root@df176dd75bd4 /]# cat /sys/fs/cgroup/cpu/cpu.shares #查看结果：
```

```
512
```

注：稍后，我们启动多个容器，测试一下是不是只能使用 512 份额的 cpu 资源。单独一个容器，看不出来使用的 cpu 的比例。因没有 docker 实例同此 docker 实例竞争。

总结：

通过 `-c` 设置的 cpu share 并不是 CPU 资源的绝对数量，而是一个相对的权重值。某个容器最终能分配到的 CPU 资源取决于它的 cpu share 占所有容器 cpu share 总和的比例。通过 cpu share 可以设置容器使用 CPU 的优先级。

比如在 host 中启动了两个容器：

```
docker run --name "container_A" -c 1024 ubuntu
```

```
docker run --name "container_B" -c 512 ubuntu
```

container_A 的 cpu share 1024，是 container_B 的两倍。当两个容器都需要 CPU 资源时，container_A 可以得到的 CPU 是 container_B 的两倍。

需要注意的是，这种按权重分配 CPU 只会发生在 CPU 资源紧张的情况下。如果 container_A 处于空闲状态，为了充分利用 CPU 资源，container_B 也可以分配到全部可用的 CPU。

17.1.2 CPU core 核心控制

参数： `--cpuset` 可以绑定 CPU

对多核 CPU 的服务器，docker 还可以控制容器运行限定使用哪些 cpu 内核和内存节点，即使用 `--cpuset-cpus` 和 `--cpuset-mems` 参数。对具有 NUMA 拓扑（具有多 CPU、多内存节点）的服务器尤其有用，可以对需要高性能计算的容器进行性能最优的配置。如果服务器只有一个内存节点，则 `--cpuset-mems` 的配置基本上不会有明显效果。

扩展：

服务器架构一般分：SMP、NUMA、MPP 体系结构介绍

从系统架构来看，目前的商用服务器大体可以分为三类：

1. 即对称多处理器结构(SMP : Symmetric Multi-Processor) 例：x86 服务器，双路服务

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

器。主板上有两个物理 cpu

2. 非一致存储访问结构 (NUMA : Non-Uniform Memory Access) 例: IBM 小型机 pSeries 690

3. 海量并行处理结构 (MPP : Massive ParallelProcessing) 。 例: 大型机 Z14

17.1.3 CPU 配额控制参数的混合使用

在上面这些参数中, cpu-shares 控制只发生在容器竞争同一个 cpu 的时间片时有效。

如果通过 cpuset-cpus 指定容器 A 使用 cpu 0, 容器 B 只是用 cpu1, 在主机上只有这两个容器使用对应内核的情况, 它们各自占用全部的内核资源, cpu-shares 没有明显效果。

如何才能有效果?

容器 A 和容器 B 配置上 cpuset-cpus 值并都绑定到同一个 cpu 上, 然后同时抢占 cpu 资源, 就可以看出效果了。

例 1: 测试 cpu-shares 和 cpuset-cpus 混合使用运行效果, 就需要一个压缩测试工具 stress 来让容器实例把 cpu 跑满。

如何把 cpu 跑满?

如何把 4 核心的 cpu 中第一和第三核心跑满? 可以运行 stress, 然后使用 taskset 绑定一下 cpu。

先扩展: stress 命令

概述: linux 系统压力测试软件 Stress。

```
[root@xianchaomaster1 ~]# yum install -y epel-release
```

```
[root@xianchaomaster1 ~]# yum install stress -y
```

stress 参数解释

-? 显示帮助信息

-v 显示版本号

-q 不显示运行信息

-n 显示已完成的指令情况

-t --timeout N 指定运行 N 秒后停止

--backoff N 等待 N 微妙后开始运行

-c 产生 n 个进程 : 每个进程都反复不停的计算随机数的平方根, 测试 cpu

-i 产生 n 个进程 : 每个进程反复调用 sync(), sync() 用于将内存上的内容写到硬盘上, 测试磁盘 io

-m --vm n 产生 n 个进程, 每个进程不断调用内存分配 malloc() 和内存释放 free() 函数, 测试内存

--vm-bytes B 指定 malloc 时内存的字节数 (默认 256MB)

--vm-hang N 指定在 free 栈的秒数

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

-d --hadd n 产生 n 个执行 write 和 unlink 函数的进程
 --hadd-bytes B 指定写的字节数
 --hadd-noclean 不 unlink

注：时间单位可以为秒 s，分 m，小时 h，天 d，年 y，文件大小单位可以为 K，M，G

例 1：产生 2 个 cpu 进程，2 个 io 进程，20 秒后停止运行

```
[root@xianchaomaster1]# stress -c 2 -i 2 --verbose --timeout 20s
```

#如果执行时间为分钟，改 20s 为 1m

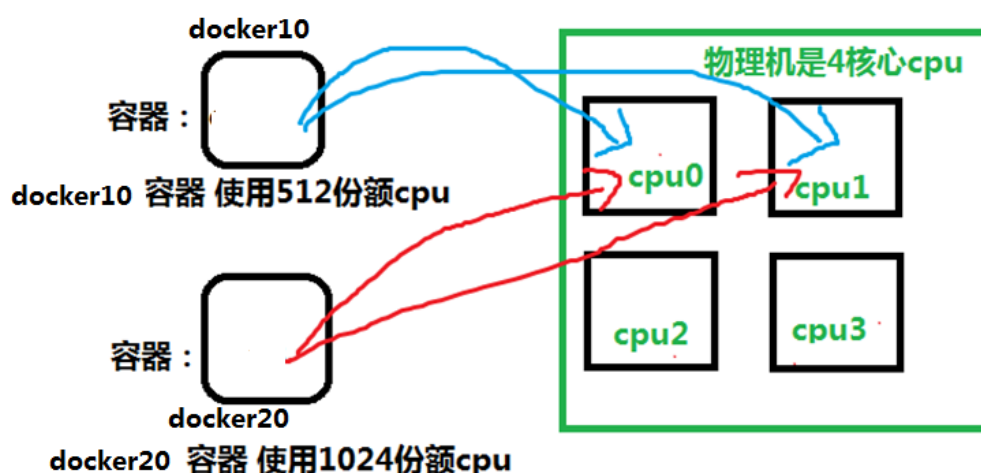
查看：

top

top - 00:46:19 up 1:27, 4 users, load average: 1.03, 0.25, 0.12										
Tasks: 414 total, 6 running, 408 sleeping, 0 stopped, 0 zombie										
%Cpu(s): 52.1 us, 38.1 sy, 0.0 ni, 9.6 id, 0.2 wa, 0.0 hi, 0.0 si, 0.0 st										
KiB Mem : 3941508 total, 2504548 free, 568944 used, 868016 buff/cache										
KiB Swap: 1023996 total, 1023996 free, 0 used. 3102552 avail Mem										
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+ COMMAND
9705	root	20	0	7260	92	0	R	99.7	0.0	0:21.99 stress
9703	root	20	0	7260	92	0	R	99.3	0.0	0:21.97 stress
9704	root	20	0	7260	92	0	R	78.8	0.0	0:17.72 stress
9706	root	20	0	7260	92	0	R	78.5	0.0	0:17.72 stress
7456	root	20	0	0	0	0	S	3.6	0.0	0:01.03 kworker/u128

例 1：测试 cpuset-cpus 和 cpu-shares 混合使用运行效果，就需要一个压缩力测试工具 stress 来让容器实例把 cpu 跑满。当跑满后，会不会去其他 cpu 上运行。如果没有在其他 cpu 上运行，说明 cgroup 资源限制成功。

实例 3：创建两个容器实例:docker10 和 docker20。让 docker10 和 docker20 只运行在 cpu0 和 cpu1 上，最终测试一下 docker10 和 docker20 使用 cpu 的百分比。实验拓扑图如下：



运行两个容器实例

```
[root@xianchaomaster1 ~]# docker run -itd --name docker10 --cpuset-cpus 0,1 --cpu-shares 512 centos /bin/bash
```

#指定 docker10 只能在 cpu0 和 cpu1 上运行，而且 docker10 的使用 cpu 的份额 512

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

#参数-itd 就是又能打开一个伪终端，又可以在后台运行着 docker 实例

```
[root@xianchaomaster1 ~]# docker run -itd --name docker20 --cpuset-cpus 0,1 --cpu-shares 1024 centos /bin/bash
```

#指定 docker20 只能在 cpu0 和 cpu1 上运行，而且 docker20 的使用 cpu 的份额 1024，比 dcker10 多一倍

测试1： 进入 docker10，使用 stress 测试进程是不是只在 cpu0,1 上运行：

```
[root@xianchaomaster1 ~]# docker exec -it docker10 /bin/bash
[root@d1a431815090 /]# yum install -y epel-release #安装 epel 扩展源
[root@d1a431815090 /]# yum install stress -y #安装 stress 命令
[root@d1a431815090 /]# stress -c 2 -v -t 10m #运行 2 个进程，把两个 cpu 占满
```

在物理机另外一个虚拟终端上运行 top 命令，按 1 快捷键，查看每个 cpu 使用情况：

```
top - 01:14:57 up 1:55, 5 users, load average: 0.33, 0.26, 0.19
Tasks: 425 total, 3 running, 422 sleeping, 0 stopped, 0 zombie
%Cpu0  :100.0 us, 0.0 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Cpu1  :100.0 us, 0.0 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Cpu2   : 1.2 us, 1.8 sy, 0.0 ni, 97.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Cpu3   : 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 3941508 total, 2209168 free, 587824 used, 1144516 buff/cache
KiB Swap: 1023996 total, 1023996 free, 0 used. 3078988 avail Mem
```

可看到正常。只在 cpu0,1 上运行

测试2： 然后进入 docker20，使用 stress 测试进程是不是只在 cpu0,1 上运行，且 docker20 上运行的 stress 使用 cpu 百分比是 docker10 的 2 倍

```
[root@xianchaomaster1 ~]# docker exec -it docker20 /bin/bash
[root@d1a431815090 /]# yum install -y epel-release #安装 epel 扩展源
[root@d1a431815090 /]# yum install stress -y
[root@f24e75bca5c0 /]# stress -c 2 -v -t 10m
```

在另外一个虚拟终端上运行 top 命令，按 1 快捷键，查看每个 cpu 使用情况：

```
top - 08:38:56 up 1:47, 4 users, load average: 2.16, 0.91, 0.39
Tasks: 468 total, 5 running, 463 sleeping, 0 stopped, 0 zombie
%Cpu0  :100.0 us, 0.0 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Cpu1  :100.0 us, 0.0 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Cpu2   : 0.0 us, 0.3 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
%Cpu3   : 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 3941508 total, 2262164 free, 477936 used, 1201408 buff/cache
KiB Swap: 1023996 total, 1023996 free, 0 used. 3131348 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
7322	root	20	0	7260	96	0	R	66.8	0.0	0:27.50	stress
7323	root	20	0	7260	96	0	R	66.4	0.0	0:28.55	stress
7325	root	20	0	7260	92	0	R	33.2	0.0	0:12.37	stress
7326	root	20	0	7260	92	0	R	33.2	0.0	0:12.37	stress

注：两个容器只在 cpu0,1 上运行，说明 cpu 绑定限制成功。而 docker20 是 docker10 使用 cpu 的 2 倍。说明--cpu-shares 限制资源成功。

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

17.2 docker 容器控制内存

Docker 提供参数 `-m`, `--memory=""` 限制容器的内存使用量。

例 1: 允许容器使用的内存上限为 128M:

```
[root@xianchaomaster1 ~]# docker run -it -m 128m centos
```

查看:

```
[root@40bf29765691 /]# cat /sys/fs/cgroup/memory/memory.limit_in_bytes
134217728
```

注: 也可以使用 `truss` 进行测试, 到现在, 我可以限制 docker 实例使用 `cpu` 的核心数和权重, 可以限制内存大小。

例 2: 创建一个 docker, 只使用 2 个 `cpu` 核心, 只能使用 128M 内存

```
[root@xianchaomaster1 ~]# docker run -it --cpuset-cpus 0,1 -m 128m centos
```

17.3 docker 容器控制 IO

```
[root@xianchaomaster1 ~]# docker run --help | grep write-b
--device-write-bps value      Limit write rate (bytes per second) to a device
                              (default [])
```

#限制此设备上的写速度 (bytes per second), 单位可以是 kb、mb 或者 gb。

```
--device-read-bps value      #限制此设备上的读速度 (bytes per second), 单位可以是 kb、mb 或者 gb。
```

情景: 防止某个 Docker 容器吃光你的磁盘 I / O 资源

例 1: 限制容器实例对硬盘的最高写入速度设定为 2MB/s。

`--device` 参数: 将主机设备添加到容器

```
[root@xianchaomaster1 ~]# mkdir -p /var/www/html/
[root@xianchaomaster1 ~]# docker run -it -v /var/www/html:/var/www/html --device
/dev/sda:/dev/sda --device-write-bps /dev/sda:2mb centos /bin/bash
```

```
[root@bd79042dbdc9 /]# time dd if=/dev/sda of=/var/www/html/test.out bs=2M count=50
oflag=direct,nonblock
```

注: `dd` 参数:

`direct`: 读写数据采用直接 IO 方式, 不走缓存。直接从内存写硬盘上。

`nonblock`: 读写数据采用非阻塞 IO 方式, 优先写 `dd` 命令的数据

```
50+0 records in
```

```
50+0 records out
```

```
52428800 bytes (52 MB) copied, 50.1831 s, 2.0 MB/s
```

```
real 0m50.201s
```

```
user 0m0.001s
```

```
sys 0m0.303s
```

注: 发现 1 秒写 2M。 限制成功。

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

18、docker 容器运行结束自动释放资源

```
[root@xianchaomaster1 ~]# docker run --help | grep rm
```

--rm 参数: Automatically remove the container when it exits

作用: 当容器命令运行结束后, 自动删除容器, 自动释放资源

例:

```
[root@xianchaomaster1 ~]# docker run -it --rm --name xianchao centos sleep 6
```

物理上查看:

```
[root@xianchaomaster1 ~]# docker ps -a | grep xianchao
```

```
6c75a9317a6b      centos              "sleep 6"          6 seconds ago      Up 4
seconds                                     mk
```

等 5s 后, 再查看:

```
[root@xianchaomaster1 ~]# docker ps | grep xianchao #自动删除了
```

18、docker 私有镜像仓库 harbor

Harbor 介绍

Docker 容器应用的开发和运行离不开可靠的镜像管理, 虽然 Docker 官方也提供了公共的镜像仓库, 但是从安全和效率等方面考虑, 部署我们私有环境内的 Registry 也是非常必要的。Harbor 是由 VMware 公司开源的企业级的 Docker Registry 管理项目, 它包括权限管理 (RBAC)、LDAP、日志审核、管理界面、自我注册、镜像复制和中文支持等功能。

官网地址: <https://github.com/goharbor/harbor>

实验环境:

安装 harbor 的机器, 主机名设置成 harbor

机器需要的内存至少要 2G, 我分配的是 4G

机器 ip: 192.168.40.181

4vCPU/4G 内存/100G 硬盘

18.1 为 Harbor 自签发证书

```
[root@192 ~]# hostnamectl set-hostname harbor && bash
```

```
[root@harbor ~]# mkdir /data/ssl -p
```

```
[root@harbor ~]# cd /data/ssl/
```

生成 ca 证书:

```
openssl genrsa -out ca.key 3072
```

#生成一个 3072 位的 key, 也就是私钥

```
openssl req -new -x509 -days 3650 -key ca.key -out ca.pem
```

#生成一个数字证书 ca.pem, 3650 表示证书的有效时间是 3 年, 按箭头提示填写即可, 没有箭头标注的为空:

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

```
[root@harbor ssl]# openssl req -new -x509 -days 3650 -key ca.key -out ca.pem
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [XX]:CH
State or Province Name (full name) []:BJ
Locality Name (eg, city) [Default City]:BJ
Organization Name (eg, company) [Default Company Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (eg, your name or your server's hostname) []:
Email Address []:
```

生成域名的证书:

```
openssl genrsa -out harbor.key 3072
```

#生成一个 3072 位的 key, 也就是私钥

```
openssl req -new -key harbor.key -out harbor.csr
```

#生成一个证书请求, 一会签发证书时需要的, 标箭头的按提示填写, 没有箭头标注的为空:

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [XX]:CH
State or Province Name (full name) []:BJ
Locality Name (eg, city) [Default City]:BJ
Organization Name (eg, company) [Default Company Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (eg, your name or your server's hostname) []:harbor
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

主机名

签发证书:

```
openssl x509 -req -in harbor.csr -CA ca.pem -CAkey ca.key -CAcreateserial -out harbor.pem -
days 3650
```

显示如下, 说明证书签发好了:

```
Signature ok
subject=/C=CH/ST=BJ/L=BJ/O=Default Company Ltd/CN=harbor
Getting CA Private Key
```

18.2 安装 Harbor

18.2.1 安装 Docker

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

关闭防火墙

```
[root@ harbor~]# systemctl stop firewalld && systemctl disable firewalld
```

关闭 iptables 防火墙

```
[root@ harbor~]# yum install iptables-services -y #安装 iptables
```

禁用 iptables

```
root@ harbor~]# service iptables stop && systemctl disable iptables
```

清空防火墙规则

```
[root@ harbor~]# iptables -F
```

关闭 selinux

```
[root@ harbor~]# setenforce 0
```

```
[root@harbor~]# sed -i 's/SELINUX=enforcing/SELINUX=disabled/g' /etc/selinux/config
```

注意: 修改 selinux 配置文件之后, 重启机器, selinux 才能永久生效

#配置时间同步

```
[root@harbor~]# yum install -y ntp ntpdate
```

```
[root@xianchaomaster1 ~]# ntpdate cn.pool.ntp.org
```

#编写计划任务

```
[root@harbor~]# crontab -e
```

```
* * /1 * * * /usr/sbin/ntpdate cn.pool.ntp.org
```

重启 crond 服务使配置生效:

```
[root@xianchaomaster1 ~]# systemctl restart crond
```

配置 hosts 文件

```
[root@xianchaomaster1 ~]# cat /etc/hosts
```

```
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
```

```
:::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
```

```
192.168.40.180 xianchaomaster1
```

```
192.168.40.181 harbor
```

```
[root@harbor harbor]# cat /etc/hosts
```

```
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
```

```
:::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
```

```
192.168.40.180 xianchaomaster1
```

```
192.168.40.181 harbor
```

安装基础软件包

```
[root@ harbor~]# yum install -y wget net-tools nfs-utils lrzsz gcc gcc-c++ make cmake  
libxml2-devel openssl-devel curl curl-devel unzip sudo ntp libaio-devel wget vim ncurses-  
devel autoconf automake zlib-devel python-devel epel-release openssh-server  
socat ipvsadm conntrack
```

版权声明, 本文档全部内容 & 版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

```
#安装 docker-ce
配置 docker-ce 国内 yum 源（阿里云）
[root@ harbor~]# yum-config-manager --add-repo http://mirrors.aliyun.com/docker-
ce/linux/centos/docker-ce.repo
```

安装 docker 依赖包

```
[root@ harbor~]# yum install -y yum-utils device-mapper-persistent-data lvm2
```

安装 docker-ce

```
[root@ harbor~]# yum install docker-ce -y
```

#启动 docker 服务

```
[root@ harbor~]# systemctl start docker && systemctl enable docker
```

```
[root@ harbor~]# systemctl status docker
```

● docker.service - Docker Application Container Engine

Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; vendor preset: disabled)

Active: active (running) since Thu 2021-07-01 21:29:18 CST; 30s ago

Docs: <https://docs.docker.com>

看到 running，表示 docker 正常运行

#查看 Docker 版本信息

```
[root@ harbor~]# docker version
```

18.2.2 开启包转发功能和修改内核参数

内核参数修改：br_netfilter 模块用于将桥接流量转发至 iptables 链，br_netfilter 内核参数需要开启转发。

```
[root@ harbor~]# modprobe br_netfilter
```

```
[root@ harbor~]# cat > /etc/sysctl.d/docker.conf <<EOF
```

```
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.bridge.bridge-nf-call-iptables = 1
```

```
net.ipv4.ip_forward = 1
```

```
EOF
```

```
[root@harbor ~]# sysctl -p /etc/sysctl.d/docker.conf
```

注：

Docker 安装后出现：WARNING: bridge-nf-call-iptables is disabled 的解决办法：

```
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.bridge.bridge-nf-call-iptables = 1
```

```
net.ipv4.ip_forward = 1;
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追责。

将 Linux 系统作为路由或者 VPN 服务就必须开启 IP 转发功能。当 linux 主机有多个网卡时一个网卡收到的信息是否能够传递给其他的网卡 如果设置成 1 的话 可以进行数据包转发，可以实现 VxLAN 等功能。不开启会导致 docker 部署应用无法访问。

#重启 docker

```
[root@xianchaomaster1 ~]# systemctl restart docker
```

```
[root@xianchaomaster1 ~]# scp /etc/docker/daemon.json 192.168.40.181:/etc/docker/
```

```
[root@harbor ~]# systemctl daemon-reload
```

```
[root@harbor ~]# systemctl restart docker
```

18.2.4 安装 harbor

创建安装目录

```
[root@harbor ssl]# mkdir /data/install -p
```

```
[root@harbor ssl]# cd /data/install/
```

安装 harbor

/data/ssl 目录下有如下文件：

ca.key ca.pem ca.srl harbor.csr harbor.key harbor.pem

```
[root@harbor install]# cd /data/install/
```

#把 harbor 的离线包 harbor-offline-installer-v2.3.0-rc3.tgz 上传到这个目录，离线包在课件里提供了

下载 harbor 离线包的地址：

<https://github.com/goharbor/harbor/releases/tag/>

解压：

```
[root@harbor install]# tar zxvf harbor-offline-installer-v2.3.0-rc3.tgz
```

```
[root@harbor install]# cd harbor
```

```
[root@harbor harbor]# cp harbor.yml.tpl harbor.yml
```

```
[root@harbor harbor]# vim harbor.yml
```

修改配置文件：

hostname: harbor

#修改 hostname，跟上面签发的证书域名保持一致

#协议用 https

certificate: /data/ssl/harbor.pem

private_key: /data/ssl/harbor.key

邮件和 ldap 不需要配置，在 harbor 的 web 界面可以配置

其他配置采用默认即可

修改之后保存退出

注：harbor 默认的账号密码：admin/Harbor12345

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

安装 docker-compose

上传课件里的 docker-compose-Linux-x86_64 文件到 harbor 机器

```
[root@harbor harbor]# mv docker-compose-Linux-x86_64 /usr/bin/docker-compose
```

```
[root@harbor harbor]# chmod +x /usr/bin/docker-compose
```

注： docker-compose 项目是 Docker 官方的开源项目，负责实现对 Docker 容器集群的快速编排。
Docker-Compose 的工程配置文件默认为 docker-compose.yml，Docker-Compose 运行目录下的必要有一个 docker-compose.yml。docker-compose 可以管理多个 docker 实例。

安装 harbor 需要的离线镜像包 docker-harbor-2-3-0.tar.gz 在课件，可上传到 harbor 机器，通过
docker load -i 解压

```
[root@harbor install]# docker load -i docker-harbor-2-3-0.tar.gz
```

```
[root@harbor install]# cd /data/install/harbor
```

```
[root@harbor harbor]# ./install.sh
```

看到下面内容，说明安装成功：

```
[Step 5]: starting Harbor ...
```

```
Creating network "harbor_harbor" with the default driver
```

```
Creating harbor-log ... done
```

```
Creating registryctl ... done
```

```
Creating harbor-db ... done
```

```
Creating redis ... done
```

```
Creating registry ... done
```

```
Creating harbor-portal ... done
```

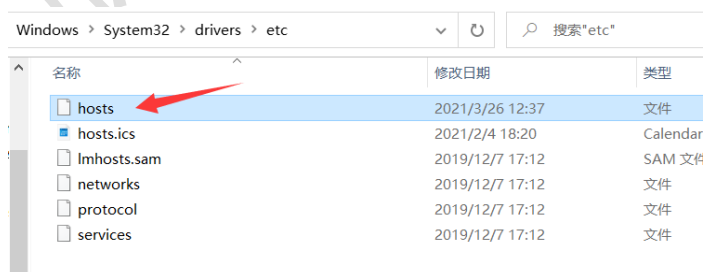
```
Creating harbor-core ... done
```

```
Creating harbor-jobservice ... done
```

```
Creating nginx ... done
```

```
✓ ---Harbor has been installed and started successfully.---
```

在自己电脑修改 hosts 文件



在 hosts 文件添加如下一行，然后保存即可

```
192.168.40.181 harbor
```

版权声明，本文档全部内容及版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

扩展:

如何停掉 harbor:

```
[root@harbor harbor]# cd /data/install/harbor
```

```
[root@harbor harbor]# docker-compose stop
```

如何启动 harbor:

```
[root@harbor harbor]# cd /data/install/harbor
```

```
[root@harbor harbor]# docker-compose start
```

如果 docker-compose start 启动 harbor 之后, 还是访问不了, 那就需要重启虚拟机

18.3 Harbor 图像化界面使用说明

在浏览器输入:

<https://harbor>



接收风险并继续, 出现如下界面, 说明访问正常



账号: admin

密码: Harbor12345

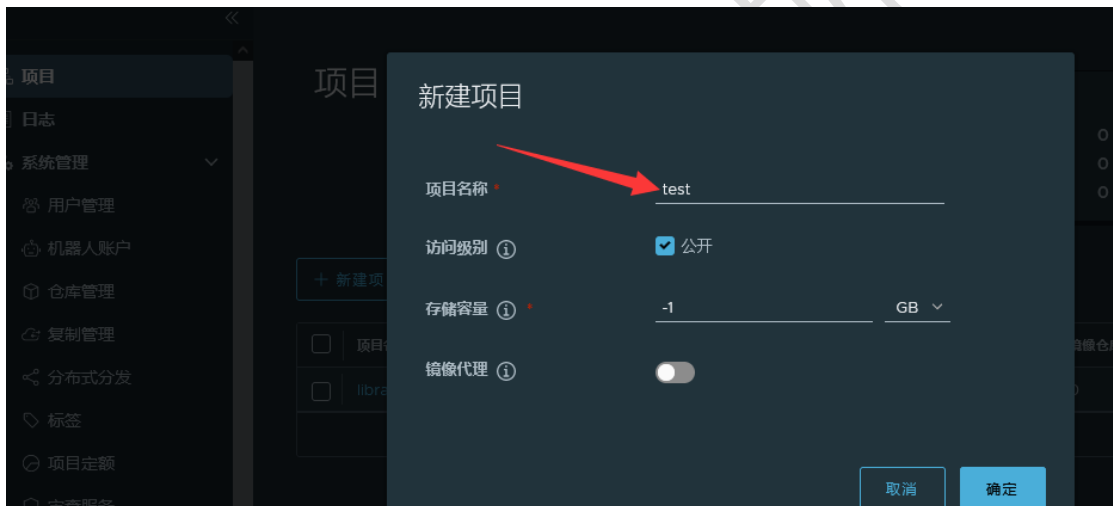
输入账号密码出现如下:

版权声明, 本文档全部内容及版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。



所有基础镜像都会放在 library 里面，这是一个公开的镜像仓库

新建项目->起个项目名字 test（把访问级别公开那个选中，让项目才可以被公开使用）



18.4 测试使用 harbor 私有镜像仓库

#修改 docker 配置

版权声明，本文档全部内容版权归韩先超所有，只可用于自己学习使用，禁止私自传阅，违者依法追究。

```
[root@xianchaomaster1 ~]# vim /etc/docker/daemon.json
```

```
{  "registry-mirrors": ["https://rsbud4vc.mirror.aliyuncs.com", "https://registry.docker-
cn.com", "https://docker.mirrors.ustc.edu.cn", "https://dockerhub.azk8s.cn", "http://hub-
mirror.c.163.com"],
  "insecure-registries": ["192.168.40.181", "harbor"]
}
```

修改配置之后使配置生效:

```
[root@xianchaomaster1 ~]# systemctl daemon-reload && systemctl restart docker
```

#查看 docker 是否启动成功

```
[root@xianchaomaster1 ~]# systemctl status docker
```

#显示如下, 说明启动成功:

Active: active (running) since Fri ... ago

注意:

配置新增加了一行内容如下:

```
"insecure-registries":["192.168.40.181"],
```

上面增加的内容表示我们内网访问 harbor 的时候走的是 http, 192.168.40.181 是安装 harbor 机器的 ip

登录 harbor:

```
[root@xianchaomaster1]# docker login 192.168.40.181
```

Username: admin

Password: Harbor12345

输入账号密码之后看到如下, 说明登录成功了:

Login Succeeded

#导入 tomcat 镜像, tomcat.tar.gz 在课件里

```
[root@xianchaomaster1 ~]# docker load -i tomcat.tar.gz
```

#把 tomcat 镜像打标签

```
[root@xianchaomaster1 ~]# docker tag tomcat:latest 192.168.40.181/test/tomcat:v1
```

执行上面命令就会把 192.168.40.181/test/tomcat:v1 上传到 harbor 里的 test 项目下

```
[root@xianchaomaster1 ~]# docker push 192.168.40.181/test/tomcat:v1
```

执行上面命令就会把 192.168.40.181/test/tomcat:v1 上传到 harbor 里的 test 项目下



版权声明, 本文档全部内容版权归韩先超所有, 只可用于自己学习使用, 禁止私自传阅, 违者依法追责。

18.5 从 harbor 仓库下载镜像

在 xianchaomaster1 机器上删除镜像

```
[root@xianchaomaster1 ~]# docker rmi -f 192.168.40.181/test/tomcat:v1
```

拉取镜像

```
[root@xianchaomaster1 ~]# docker pull 192.168.40.181/test/tomcat:v1
```